What Drives Childhood Obesity?

An Explainable Machine Learning Analysis of London Wards

Preparation

- Github link
- Number of words: 1493
- Runtime: 305 seconds (*Memory 10 GB, CPU Intel i7-10700 CPU @2.90GHz*)
- Coding environment: Podman
- License: this notebook is made available under the Creative Commons Attribution license.
- Additional library *[libraries not included in SDS Docker or not used in this module]*:
  - **xgboost-3.2.0**
  - **shap-0.51.0**

Table of contents

1. Introduction
2. Research questions
3. Data
4. Methodology
5. Results and discussion
6. Conclusion
7. References

Introduction

[go back to the top]

```
import time
start_time = time.time()
```

```
import pandas as pd
pd.set_option('display.float_format', lambda x: '%.4f' % x)
import numpy as np
from matplotlib import pyplot
import matplotlib.pyplot as plt
import seaborn as sns
import geopandas as gpd
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

Childhood obesity has emerged as a pressing global public health challenge, closely linked to severe lifelong comorbidities such as type II diabetes (Colmenarejo, 2020). In the UK, prevalence varies significantly across neighbourhoods and worsens dramatically with age, rising from approximately 10.5% in Reception to over 22.2% by Year 6 (NCMP, 2025). This issue is particularly acute in urban settings like London.

To mitigate this crisis, developing robust predictive models using multidimensional risk factors is crucial. Traditionally, statistical approaches such as linear and logistic regression have been widely employed to evaluate early life risk factors (Pei et al., 2013). However, childhood obesity is a complex condition driven by interacting individual and area-level characteristics, prompting a shift towards Machine Learning (ML) models capable of capturing nonlinear relationships. For instance, Hammond et al. (2019) demonstrated the superiority of Gradient Boosting over traditional regression, while Jeong et al. (2023) successfully utilised Random Forest coupled with SHAP to interpret social and environmental influences.

Despite these advancements, a critical research gap remains. Existing studies predominantly focus on single age cohorts or rely heavily on individual-level clinical data, with little comparative analysis of how area-level socioeconomic and built environment factors shift in predictive importance across developmental stages. Addressing this, the study employs MLR, RF, XGBoost, and SHAP to compare how ward-level characteristics drive childhood obesity differently between Reception and Year 6 cohorts in London.

Research questions

[go back to the top]

RQ1 | To what extent can multidimensional features (socioeconomic, demographic, health, and environmental exposure factors) accurately predict the obesity rates of children in different school years at the London ward level?

RQ2 | Within the optimal predictive model, which key features act as the most significant drivers of childhood obesity, and how do their marginal effects differ between the two age groups?

Data

[go back to the top]

Data Source

The dependent variable is sourced from the London Ward-level Prevalence of Childhood Obesity dataset (2010/11–2012/13), which provides overweight and obesity prevalence for Reception and Year 6 cohorts across over 600 wards.

For independent variables, the London Ward Atlas was utilized due to its comprehensive coverage of area-level statistics. Due to privacy constraints associated with individual clinical data, this study focuses exclusively on macroscopic neighborhood characteristics.

Feature Selection

Guided by recent literature, feature selection was conducted across four dimensions: **demographic, socioeconomic, health condition, and environmental exposure**. While Mazza et al. (2025) emphasize the predictive power of standard socioeconomic and demographic indicators, this study further incorporates environmental and health behavior metrics to broaden the analytical scope, an approach supported by Salerno et al. (2024).

This theoretically driven selection yielded 23 initial predictors.

Variable	Type	Description	Year
Target Variables			
Childhood obesity rate: reception (aged 4-5 years) and year 6 (aged 10-11 years)	Numeric	The children obesity rate of wards	2010/11-2012/13

Variable	Type	Description	Year
Demographic Features			
Median age	Numeric	Median age of wards	2011
Population under 15 years	Numeric	Percentage of population age 15 years or younger	2013
Population density	Numeric	Population density (persons per sq km)	2011
White population	Numeric	Percentage of White population	2011
Population borned in UK	Numeric	Percentage of population borned in UK	2011
Socioeconomic Features			
Median house price	Numeric	The median house price (UK pounds) of wards	2011
Homeownership	Numeric	Percentage of owner-occupied housing units	2011
Median household income	Numeric	The income (UK pound) where half of households in a county earn more and half of households earn less	2012
Children in Single-Parent Household	Numeric	Percentage of children that live in a household headed by a single parent	2011
Parents unempolyment	Numeric	Percentage of households with no adults in employment with dependent children	2011
Higher education qulifications	Numeric	Percentage of people with level 4 qulifications and above	2011
Housing benefits claimants	Numeric	Percentage of households that claim housing benefits	2011
Children in poverty	Numeric	Percentage of people under age 16 in poverty	2011
Crime rate	Numeric	Crime rate of wards	2011
Indices of Deprivation	Numeric	Average scores of Indices of Deprivation	2010
Income Deprivation Affecting Children Index (IDACI)	Numeric	Average scores of IDACI	2010
Health Condition Features			
Life expectancy	Numeric	Average life expectancy at birth of all genders	2009-2013
Subjective well-being average score	Numeric	Subjective well-being average score of wards	2011-2013
People with Bad or Very Bad Health	Numeric	Percentage of people with Bad or Very Bad Health	2011

Variable	Type	Description	Year
Environmental Exposure Features			
Air emissions	Numeric	Annual Mean of Nitrogen Oxide (NOx)	2011
Deficiency in access to green space	Numeric	Percentage of homes with deficiency in access to nature	2012
Public Transport Accessibility	Numeric	Average PTAL score	2011
Car access	Numeric	Cars per household	2011

Data Pre-processing and Presentation

The workflow was executed in the following steps:

1. *Target Formatting & Cleaning*: Converted target variables to numeric formats (coercing character errors to NaN) and removed rows with missing entries.
2. *Feature Engineering*: Computed proportional rates from absolute feature counts to standardize the scale across different wards.
3. *Data Integration*: Merged the cleaned feature set (X) and target variables (y) via an inner join based on the unique Ward Code.
4. *Exploratory Data Analysis (EDA)*: Conducted statistical summaries and spatial mapping to contextualize the data structures and variable distributions.

Prevalence of Childhood Obesity

```
# Load childhood obesity prvalance data
obesity = pd.read_csv("https://raw.githubusercontent.com/mixedmint/DSSS/refs/heads/main/data/
obesity.head(10)
```

	Code	Borough	Names	obesity rate % (age 4-5)	obesity rate % (age
0	E05000001	City of London	Aldersgate	s	s
1	E05000015	City of London	Cripplegate	s	s
2	E05000017	City of London	Farringdon Within	s	s
3	E05000021	City of London	Portsoken	s	s
4	E05000026	Barking and Dagenham	Abbey	15.1%	23.2%
5	E05000027	Barking and Dagenham	Alibon	9.6%	25.6%
6	E05000028	Barking and Dagenham	Becontree	12.1%	28.4%
7	E05000029	Barking and Dagenham	Chadwell Heath	12.1%	28.8%
8	E05000030	Barking and Dagenham	Eastbrook	10.9%	24.1%

	Code	Borough	Names	obesity rate % (age 4-5)	obesity rate % (age
9	E05000031	Barking and Dagenham	Eastbury	12.3%	24.3%

```
obesity.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 628 entries, 0 to 627
Data columns (total 5 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Code                        628 non-null    object
 1   Borough                     628 non-null    object
 2   Names                       628 non-null    object
 3   obesity rate % (age 4-5)    628 non-null    object
 4   obesity rate % (age 10-11)  628 non-null    object
dtypes: object(5)
memory usage: 24.7+ KB
```

```
# Since suppressed data is marked with `s` for privacy,
# the obesity rate columns are parsed as object instead of float.
# We will convert them to numeric types and drop any NaN rows

# First, simplify complex column names
obesity = obesity.rename({'obesity rate % (age 4-5)': 'Reception', 'obesity rate % (age 10-1

# Then convert obesity rate columns from strings to floats
cols_to_clean = ['Reception', 'Year 6']
for col in cols_to_clean:
    obesity[col] = pd.to_numeric(obesity[col].str.replace('%', ''), errors='coerce')

# Drop rows with NaN values and Borough column
obesity = obesity.dropna().drop(['Borough'], axis=1)

obesity.head()
```

	Code	Names	Reception	Year 6
4	E05000026	Abbey	15.1000	23.2000
5	E05000027	Alibon	9.6000	25.6000
6	E05000028	Becontree	12.1000	28.4000

	Code	Names	Reception	Year 6
7	E05000029	Chadwell Heath	12.1000	28.8000
8	E05000030	Eastbrook	10.9000	24.1000

```
# Check data types of obesity rates after processing
obesity.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 609 entries, 4 to 626
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Code       609 non-null    object
 1   Names      609 non-null    object
 2   Reception  609 non-null    float64
 3   Year 6     609 non-null    float64
dtypes: float64(2), object(2)
memory usage: 23.8+ KB
```

Features

```
# Load features data
raw_features = pd.read_csv("https://raw.githubusercontent.com/mixedmint/DSSS/refs/heads/main/
raw_features.head()
```

	New Code	Borough	Names	Median age	Aged 0-15	Population density	W
0	E09000001	City of London	City of London	39	7.5000	2312.5000	5
1	E05000026	Barking and Dagenham	Abbey	29	25.2747	9884.6154	3
2	E05000027	Barking and Dagenham	Alibon	32	25.9615	7464.2857	7
3	E05000028	Barking and Dagenham	Becontree	32	24.8963	8923.0769	6
4	E05000029	Barking and Dagenham	Chadwell Heath	34	24.1379	2970.5882	6

```
raw_features.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 625 entries, 0 to 624
Data columns (total 36 columns):
```

```
 #   Column                                                    Non-
Null Count  Dtype
---  ------                                                    --------------  --
 0   New Code                                                  625 non-
null    object
 1   Borough                                                   625 non-
null    object
 2   Names                                                     625 non-
null    object
 3   Median age                                                625 non-
null    int64
 4   Aged 0-15                                                 625 non-
null    float64
 5   Population density                                        625 non-
null    float64
 6   White                                                     625 non-
null    int64
 7   Mixed                                                     625 non-
null    int64
 8   Asian or Asian British                                    625 non-
null    int64
 9   Black or Black British                                    625 non-
null    int64
 10  Other                                                     625 non-
null    int64
 11  Born in UK                                                625 non-
null    int64
 12  Not Born in UK                                            625 non-
null    int64
 13  Life expectancy                                           625 non-
null    float64
 14  Subjective well-being average score                       625 non-
null    float64
 15  People with Bad or Very Bad Health                        625 non-
null    float64
 16  Median house price                                        625 non-
null    float64
 17  Owned: Owned outright                                     625 non-
null    int64
 18  Owned: Owned with a mortgage or loan (inc shared)         625 non-
null    int64
 19  Social rented                                             625 non-
null    int64
```

```
 20  Private rented                                                    625 non-
null    int64
 21  Rent Free                                                         625 non-
null    int64
 22  Median household income                                           625 non-
null    int64
 23  Lone parent                                                       625 non-
null    int64
 24  All Households                                                    625 non-
null    int64
 25  % of households with no adults in employment with dependent children  625 non-
null    float64
 26  % Highest level of qualification: Level 4 qualifications and above    625 non-
null    float64
 27  Housing benefits claimants                                        625 non-
null    float64
 28  Children in poverty                                               625 non-
null    float64
 29  Crime rate                                                        625 non-
null    float64
 30  Air emissions                                                     625 non-
null    float64
 31  % homes with deficiency in access to nature                       625 non-
null    float64
 32  Public transport accessibility                                    625 non-
null    float64
 33  Car access                                                        625 non-
null    float64
 34  Indices of Deprivation                                            625 non-
null    float64
 35  IDACI                                                             625 non-
null    float64
dtypes: float64(17), int64(16), object(3)
memory usage: 175.9+ KB
```

```python
# Calculate the percentage of White population
# Define ethnicity columns
ethnicity_cols = [
    'White',
    'Mixed',
    'Asian or Asian British',
    'Black or Black British',
```

```
    'Other'
]

# Calculate White population percentage
raw_features['White population'] = (raw_features['White'] / raw_features[ethnicity_cols].sum

# Drop the original data columns
raw_features = raw_features.drop(ethnicity_cols, axis=1)

raw_features['White population'].head()
```

```
0   78.6305
1   28.1245
2   70.0530
3   58.8047
4   62.1096
Name: White population, dtype: float64
```

```
# Calculate the percentage of population borned in UK
raw_features['Born in uk'] = (raw_features['Born in UK'] / raw_features[['Born in UK', 'Not B

# Drop the original data columns
raw_features = raw_features.drop(['Born in UK', 'Not Born in UK'], axis=1)

raw_features['Born in uk'].head()
```

```
0   63.3220
1   42.6951
2   75.3394
3   69.9437
4   75.2320
Name: Born in uk, dtype: float64
```

```
# Calculate the percentage of owner-occupied housing units (Homeownership)
# Define ownership columns
owenership_cols = [
    'Owned: Owned outright',
    'Owned: Owned with a mortgage or loan (inc shared)',
    'Social rented',
    'Private rented',
    'Rent Free'
```

```
]

# Define Owner-occupied housing columns
owned_cols = [
    'Owned: Owned outright',
    'Owned: Owned with a mortgage or loan (inc shared)'
]

# Calculate Homeownership
raw_features['Homeownership'] = (raw_features[owned_cols].sum(axis=1) / raw_features[owenersh

# Drop the original data columns
raw_features = raw_features.drop(owenership_cols, axis=1)

raw_features['Homeownership'].head()
```

```
0   42.5998
1   33.7927
2   46.5626
3   48.9362
4   54.4901
Name: Homeownership, dtype: float64
```

```
# Calculate the percentage of Children in Single-Parent Household
raw_features['Children in single-parent household'] = (raw_features['Lone parent'] / raw_feat

# Drop the original data columns
raw_features = raw_features.drop(['Lone parent', 'All Households'], axis=1)

raw_features['Children in single-parent household'].head()
```

```
0    3.4892
1   14.1732
2   20.9519
3   20.5752
4   19.6854
Name: Children in single-parent household, dtype: float64
```

```
# Simplify complex column names
raw_features = raw_features.rename({'Subjective well-being average score': 'Well-being score
```

```
                                        'People with Bad or Very Bad Health': 'People in bad heal
                                        '% of households with no adults in employment with depend
                                        '% Highest level of qualification: Level 4 qualifications
                                       '% homes with deficiency in access to nature': 'Green defi
                                       axis=1)
```

```
raw_features.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 625 entries, 0 to 624
Data columns (total 26 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   New Code                              625 non-null    object
 1   Borough                               625 non-null    object
 2   Names                                 625 non-null    object
 3   Median age                            625 non-null    int64
 4   Aged 0-15                             625 non-null    float64
 5   Population density                    625 non-null    float64
 6   Life expectancy                       625 non-null    float64
 7   Well-being score                      625 non-null    float64
 8   People in bad health                  625 non-null    float64
 9   Median house price                    625 non-null    float64
 10  Median household income               625 non-null    int64
 11  Parents unemployment                  625 non-null    float64
 12  Higher education qualifications       625 non-null    float64
 13  Housing benefits claimants            625 non-null    float64
 14  Children in poverty                   625 non-null    float64
 15  Crime rate                            625 non-null    float64
 16  Air emissions                         625 non-null    float64
 17  Green deficit                         625 non-null    float64
 18  Public transport accessibility        625 non-null    float64
 19  Car access                            625 non-null    float64
 20  Indices of Deprivation                625 non-null    float64
 21  IDACI                                 625 non-null    float64
 22  White population                      625 non-null    float64
 23  Born in uk                            625 non-null    float64
 24  Homeownership                         625 non-null    float64
 25  Children in single-parent household   625 non-null    float64
dtypes: float64(21), int64(2), object(3)
memory usage: 127.1+ KB
```

Joining the Datasets

```
# Merge two databases based on the ward code
df = pd.merge(obesity, raw_features, left_on='Code', right_on='New Code', how='inner')

# Remove duplicate and unnecessary columns
df = df.drop(['New Code', 'Borough', 'Names_y'], axis=1).rename({'Names_x': 'Names'}, axis=1)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 609 entries, 0 to 608
Data columns (total 27 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   Code                                  609 non-null    object
 1   Names                                 609 non-null    object
 2   Reception                             609 non-null    float64
 3   Year 6                                609 non-null    float64
 4   Median age                            609 non-null    int64
 5   Aged 0-15                             609 non-null    float64
 6   Population density                    609 non-null    float64
 7   Life expectancy                       609 non-null    float64
 8   Well-being score                      609 non-null    float64
 9   People in bad health                  609 non-null    float64
 10  Median house price                    609 non-null    float64
 11  Median household income               609 non-null    int64
 12  Parents unemployment                  609 non-null    float64
 13  Higher education qualifications       609 non-null    float64
 14  Housing benefits claimants            609 non-null    float64
 15  Children in poverty                   609 non-null    float64
 16  Crime rate                            609 non-null    float64
 17  Air emissions                         609 non-null    float64
 18  Green deficit                         609 non-null    float64
 19  Public transport accessibility        609 non-null    float64
 20  Car access                            609 non-null    float64
 21  Indices of Deprivation                609 non-null    float64
 22  IDACI                                 609 non-null    float64
 23  White population                      609 non-null    float64
 24  Born in uk                            609 non-null    float64
 25  Homeownership                         609 non-null    float64
 26  Children in single-parent household   609 non-null    float64
dtypes: float64(23), int64(2), object(2)
memory usage: 128.6+ KB
```

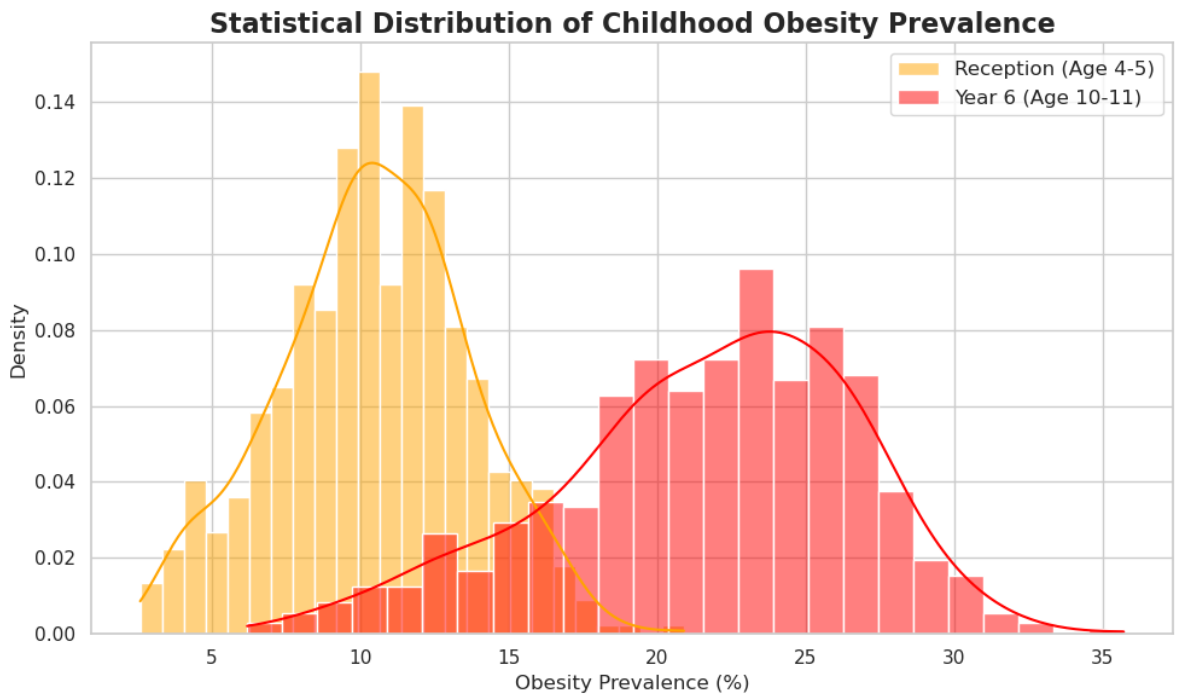
```
df.describe()
```

	Reception	Year 6	Median age	Aged 0-15	Population density	Life expectancy	Well-being sc
count	609.0000	609.0000	609.0000	609.0000	609.0000	609.0000	609.0000
mean	10.4685	21.5767	34.2463	19.7783	8014.4102	81.8959	7.6307
std	3.1981	5.0440	4.0056	3.5749	4757.4747	2.1672	0.3179
min	2.6000	6.2000	26.0000	6.8670	177.5862	75.9843	6.7803
25%	8.3000	18.5000	31.0000	17.7570	4528.5714	80.4131	7.3955
50%	10.5000	22.2000	33.0000	19.6078	6750.0000	81.8663	7.6389
75%	12.5000	25.4000	37.0000	21.8750	10735.2941	83.3154	7.8756
max	20.9000	35.7000	46.0000	32.4000	29375.0000	89.7823	8.5547

Exploratory Data Analysis (EDA)

```
# Plot statistical disttribution chart of obesity prevalence

# Set figure size and theme
plt.figure(figsize=(10, 6))
sns.set_theme(style="whitegrid")

# Reception obesity histogram+KDE
sns.histplot(
    df['Reception'],
    kde=True,
    color='orange',
    label='Reception (Age 4-5)',
    stat='density',
    alpha=0.5,
    bins=25
)

# Year 6 obesity histogram+KDE
sns.histplot(
    df['Year 6'],
    kde=True,
    color='red',
    label='Year 6 (Age 10-11)',
    stat='density',
    alpha=0.5,
    bins=25
```

```
)

# Configure title and axis labels
plt.title('Statistical Distribution of Childhood Obesity Prevalence', fontsize=16, fontweight
plt.xlabel('Obesity Prevalence (%)', fontsize=12)
plt.ylabel('Density', fontsize=12)

# Add legend
plt.legend(fontsize=12)

# Set layout and show plot
plt.tight_layout()
plt.show()
```



The rightward shift of the Year 6 peak indicates increased obesity prevalence with age, with both distributions approximately normally distributed.

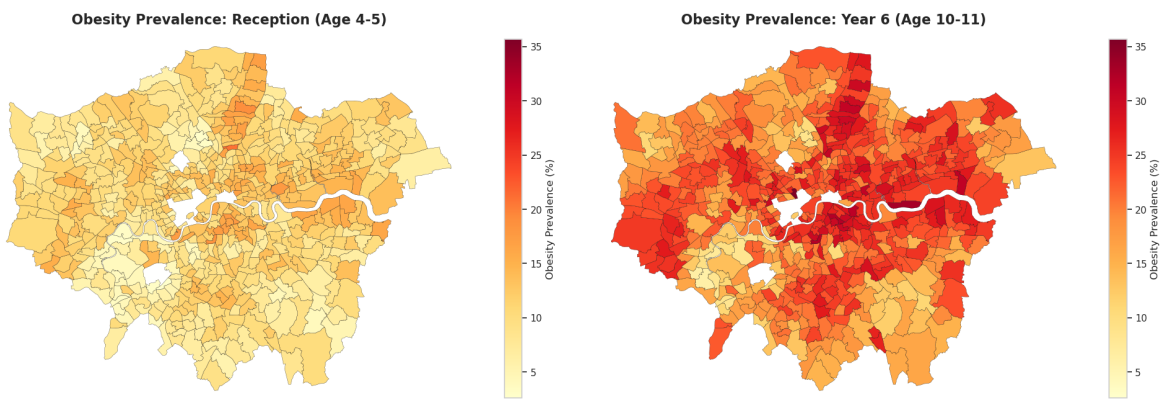
```
# Then I will plot choropleth maps of childhood obesity rates to see their spatial distributi

# I downloaded 2011 UK Ward Boundaries files from ONS
# and selected all London wards:
```

```
# uk_wards = gpd.read_file('data/Wards_December_2011_GCB.geojson')
# lookup_table = pd.read_csv('data/Wards_and_Local_Authorities_(2011)_to_Wards_and_Local_Aut
# lookup_table = lookup_table.drop_duplicates(subset=['WD11CD'])
# merged_wards = uk_wards.merge(lookup_table, left_on='wd11cd', right_on='WD11CD', how='left
# london_wards = merged_wards[merged_wards['LAD11CD'].str.startswith('E09', na=False)]
# columns_to_drop = ['WD11CD', 'WD11NM', 'WD22CD', 'WD22NM', 'LAD22CD', 'LAD22NM']
# london_wards = london_wards.drop(columns=columns_to_drop)

# Load London Ward geojson file
london_map = gpd.read_file("https://raw.githubusercontent.com/mixedmint/DSSS/refs/heads/main/
london_map.info()


# Merge london_map and dataset
geo_df = london_map.merge(df, left_on='wd11cd', right_on='Code', how='left')

# Set CRS
geo_df = geo_df.to_crs(epsg=27700)
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 649 entries, 0 to 648
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   OBJECTID  649 non-null    int32
 1   wd11cd    649 non-null    object
 2   wd11nm    649 non-null    object
 3   LAD11NM   649 non-null    object
 4   geometry  649 non-null    geometry
dtypes: geometry(1), int32(1), object(3)
memory usage: 22.9+ KB
```

```
# Plot childhood obesity prevalance choropleth maps
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))

vmin = min(geo_df['Reception'].min(), geo_df['Year 6'].min())
vmax = max(geo_df['Reception'].max(), geo_df['Year 6'].max())

plot_kwds = {
    'cmap': 'YlOrRd',
    'vmin': vmin,
```

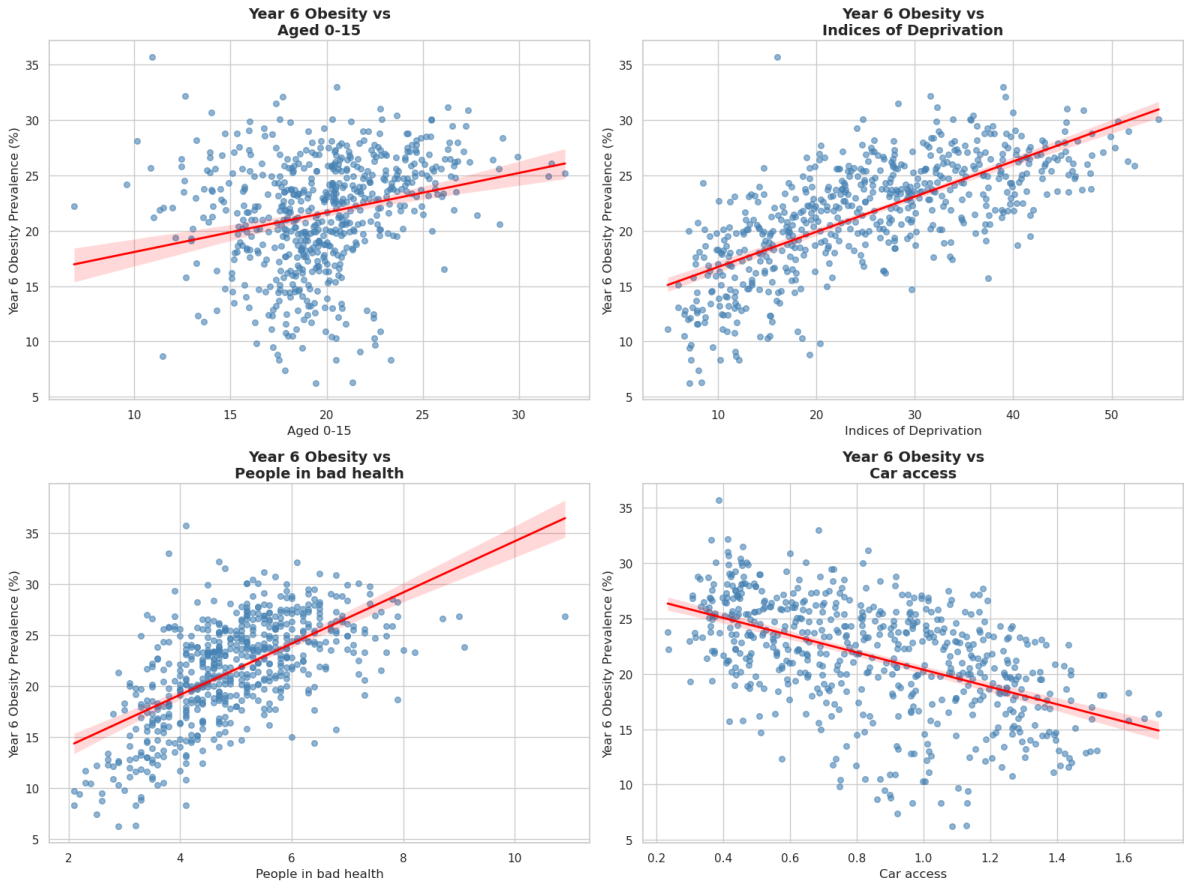
```
    'vmax': vmax,
    'edgecolor': 'black',
    'linewidth': 0.2,
    'legend': True,
    'legend_kwds': {'shrink': 0.6, 'label': 'Obesity Prevalence (%)'}
}

# Reception obesity map
geo_df.plot(column='Reception', ax=ax1, **plot_kwds)
ax1.set_title('Obesity Prevalence: Reception (Age 4-5)', fontsize=16, fontweight='bold')
ax1.axis('off')

# Year 6 obesity map
geo_df.plot(column='Year 6', ax=ax2, **plot_kwds)
ax2.set_title('Obesity Prevalence: Year 6 (Age 10-11)', fontsize=16, fontweight='bold')
ax2.axis('off')

plt.tight_layout()
plt.show()
```



The maps also reveal a stark age-related increase in obesity, with high-prevalence clusters persistently concentrated in central-eastern London wards.

```
# Plot scatter plots to explore the correlation between obesity prevalence and features

# Since we have 23 features in total
# Plotting all of them would be visually overwhelming and difficult to interpret
# Therefore, we select 4 representative features
selected_features = [
    'Aged 0-15',                # From demographic dimension
```

```
    'Indices of Deprivation',    # From socioeconomic dimension
    'People in bad health',      # From health condition dimension
    'Car access'                 # From environmental exposure dimension
]

# Set figure size and theme
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
axes = axes.flatten()

sns.set_theme(style="whitegrid")

# Iterate through each selected feature to plot its relationship with Year 6 obesity
for i, feature in enumerate(selected_features):
    sns.regplot(
        data=df,
        x=feature,
        y='Year 6',  # Using Year 6 as the target variable for demonstration as it exhibits r
        ax=axes[i],
        scatter_kws={'alpha': 0.6, 'color': 'steelblue', 's': 30},
        line_kws={'color': 'red', 'linewidth': 2}
    )

    # Configure titles and axis labels for each subplot
    axes[i].set_title(f'Year 6 Obesity vs\n{feature}', fontsize=14, fontweight='bold')
    axes[i].set_xlabel(feature, fontsize=12)
    axes[i].set_ylabel('Year 6 Obesity Prevalence (%)', fontsize=12)

plt.tight_layout()
plt.show()
```

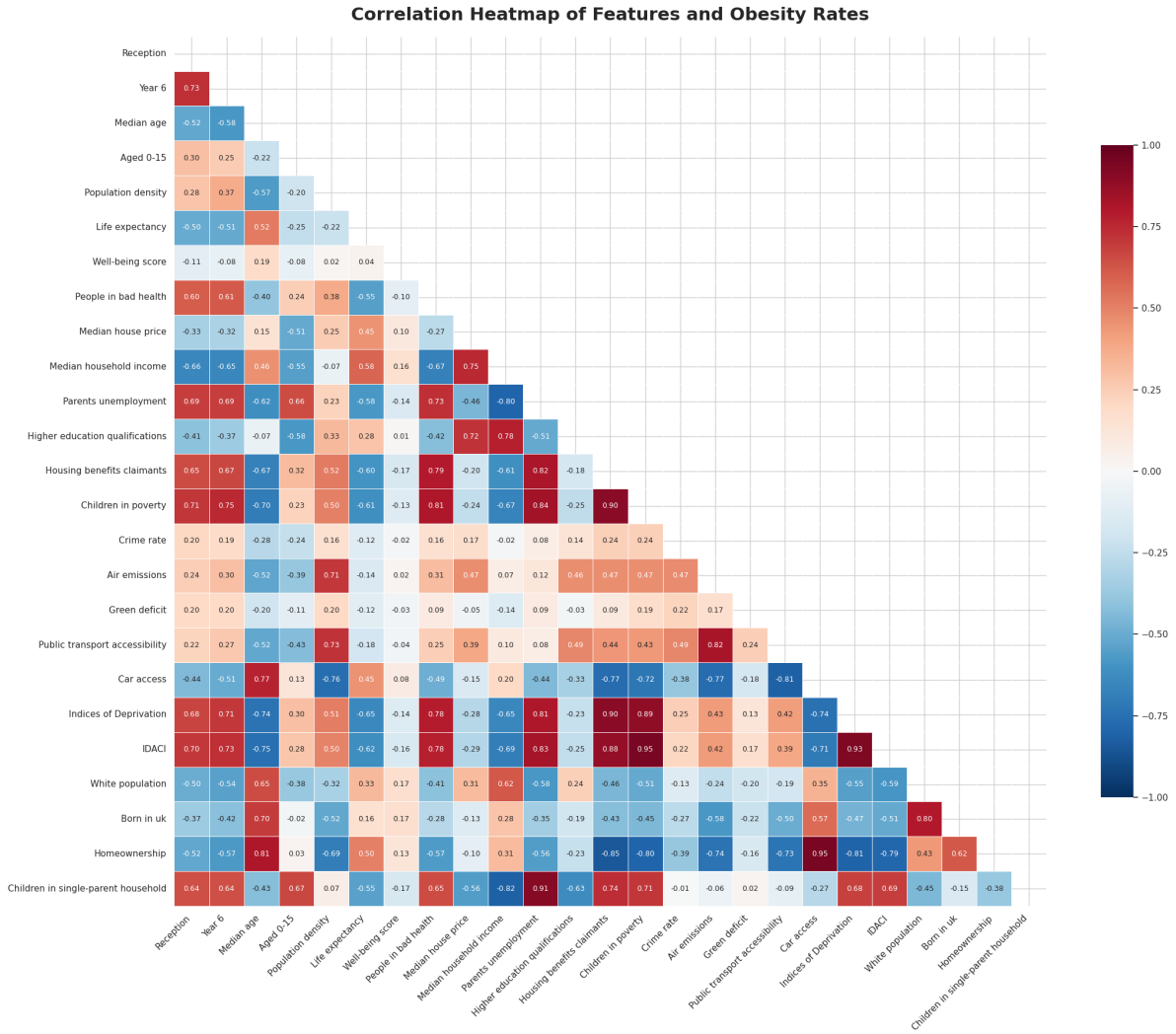


The scatter plots reveal distinct linear patterns across all four dimensions. Indices of Deprivation and People in bad health show the strongest positive associations with Year 6 obesity prevalence, while Car access exhibits a negative relationship, likely because car ownership acts as a proxy for higher socioeconomic status and lower deprivation. Aged 0-15 shows a weaker positive trend.

However, considerable scatter around all regression lines implies the presence of nonlinear dynamics and interaction effects that simple linear models cannot fully capture. **This motivates the use of MLR as an interpretable baseline, alongside RF and XGBoost to model these more complex structures.**

```
# Correlation Heatmap
# Using a lower-triangle heatmap for better readability with 23 features

# Filter numeric features (exclude strings, codes, or geometry columns)
numeric_df = df.select_dtypes(include=['float64', 'int64'])

# Compute the Pearson correlation matrix
```

```python
corr_matrix = numeric_df.corr()

# Create a mask to hide the upper triangle
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

# Set figure size
plt.figure(figsize=(22, 18))

# Plot the heatmap
sns.heatmap(
    corr_matrix,
    mask=mask,
    cmap='RdBu_r',
    annot=True,
    fmt=".2f",
    annot_kws={"size": 9},
    linewidths=0.5,
    vmin=-1, vmax=1,
    cbar_kws={"shrink": .75}
)

# Set title and axis labels
plt.title('Correlation Heatmap of Features and Obesity Rates',
          fontsize=22, fontweight='bold', pad=20)

plt.xticks(rotation=45, ha='right', fontsize=11)
plt.yticks(fontsize=11)

plt.tight_layout()
plt.show()
```



The heatmap reveals that IDACI, Indices of Deprivation and Children in poverty are the strongly correlated with obesity. However, these variables are also heavily inter-correlated. IDACI and Indices of Deprivation share a coefficient of 0.95, indicating severe **multicollinearity**. To address this, **VIF analysis is applied to iteratively remove redundant features prior to modelling**.

Methodology

[go back to the top]

This study employs a four-stage analytical pipeline to address research questions.

To address RQ1, the pipeline begins with **Variance Inflation Factor (VIF)** analysis, which iteratively removes features exceeding a threshold of 10, ensuring retained predictors are sufficiently independent and regression coefficients remain reliable. **Multiple Linear Regression (MLR)** is then applied as an interpretable linear baseline, where coefficient estimates and p-values quantify each feature's linear contribution to obesity prevalence. Subsequently, **Random Forest (RF) and XGBoost** are employed to capture nonlinear relationships and feature interactions that MLR cannot model. Both are optimised via 5-fold cross-validation with GridSearchCV, and their predictive performance (R^2, RMSE) is compared against MLR to assess whether nonlinear models offer meaningful improvements.

To address RQ2, **SHAP** is applied to the best-performing model, decomposing its predictions into individual feature contributions. This transforms the otherwise opaque tree-based models into interpretable outputs, enabling a direct comparison of the marginal effects of key drivers between the Reception and Year 6 cohorts.

Variance Inflation Factor (VIF)

```
features_list = [
    'Median age', 'Aged 0-15', 'Population density', 'Life expectancy', 'Well-being score',
    'People in bad health', 'Median house price', 'Median household income', 'Parents unempl
    'Higher education qualifications', 'Housing benefits claimants', 'Children in poverty',
    'Air emissions', 'Green deficit', 'Public transport accessibility', 'Car access',
    'Indices of Deprivation', 'IDACI', 'White population', 'Born in uk',
    'Homeownership', 'Children in single-parent household'
]

# Extract feature dataframe and drop any missing values
X = df[features_list].copy()
X = X.dropna()

# Define a function to iteratively calculate VIF and filter features
def calculate_vif_and_filter(X, threshold=5.0):
    X_with_const = add_constant(X)

    dropped_features = []

    while True:
        # Calculate VIF for all current features
        vif_data = pd.DataFrame()
        vif_data["Feature"] = X_with_const.columns
        vif_data["VIF"] = [variance_inflation_factor(X_with_const.values, i)
                           for i in range(X_with_const.shape[1])]
```

```
        # Exclude the constant term
        vif_features = vif_data[vif_data['Feature'] != 'const']

        # Identify the feature with the highest VIF
        max_vif_row = vif_features.loc[vif_features['VIF'].idxmax()]
        max_vif_value = max_vif_row['VIF']
        max_vif_feature = max_vif_row['Feature']

        # Drop the feature if its VIF exceeds the threshold
        if max_vif_value > threshold:
            print(f"Dropping: {max_vif_feature} (VIF = {max_vif_value:.2f})")
            X_with_const = X_with_const.drop(columns=[max_vif_feature])
            dropped_features.append(max_vif_feature)
        else:
            break

    # Return the final selected features, the VIF dataframe, and dropped features
    final_vif = vif_data[vif_data['Feature'] != 'const'].sort_values(by='VIF', ascending=Fals
    return list(final_vif['Feature']), final_vif, dropped_features

# Set the VIF threshold and execute the function
vif_threshold = 10.0
final_features, final_vif_df, dropped = calculate_vif_and_filter(X, threshold=vif_threshold)

# Print final results
print("VIF threshold:", vif_threshold)
print("Number of VIF-selected features:", len(final_features))
display(final_vif_df)
```

```
Dropping: Homeownership (VIF = 30.93)
Dropping: Median household income (VIF = 27.70)
Dropping: Children in poverty (VIF = 25.37)
Dropping: Car access (VIF = 20.75)
Dropping: Parents unemployment (VIF = 19.59)
Dropping: Housing benefits claimants (VIF = 14.24)
Dropping: IDACI (VIF = 10.99)
VIF threshold: 10.0
Number of VIF-selected features: 16
```

	Feature	VIF
15	Born in uk	8.1857
13	Indices of Deprivation	7.1247
14	White population	7.0609
1	Median age	5.8269
10	Air emissions	5.3133
12	Public transport accessibility	4.9229
16	Children in single-parent household	4.6785
8	Higher education qualifications	4.6367
6	People in bad health	4.4704
7	Median house price	3.8860
3	Population density	3.3636
2	Aged 0-15	3.2173
4	Life expectancy	2.3537
9	Crime rate	1.7229
11	Green deficit	1.1857
5	Well-being score	1.1731

Multiple Linear Regression (MLR)

For Reception

```
X = df[final_features]
y_rec = df['Reception']

# Drop missing values to ensure X and y maintain consistent lengths
mlr_data_rec = pd.concat([X, y_rec], axis=1).dropna()
X_clean_rec = mlr_data_rec[final_features]
y_clean_rec = mlr_data_rec['Reception']

# Split data into training (80%) and testing (20%) sets
# I asked Gemini for advice
# This establishes a fair baseline comparison against non-linear models (RF/XGBoost) later
X_train_rec, X_test_rec, y_train_rec, y_test_rec = train_test_split(
    X_clean_rec, y_clean_rec, test_size=0.2, random_state=42
)

# Build the MLR model
X_train_rec_sm = sm.add_constant(X_train_rec)
X_test_rec_sm = sm.add_constant(X_test_rec)
```

```python
mlr_rec = sm.OLS(y_train_rec, X_train_rec_sm).fit()

# Print statistical summary
print(mlr_rec.summary())

# Predict on the test set to evaluate model performance
y_pred_mlr_rec = mlr_rec.predict(X_test_rec_sm)

# Calculate and print R² and RMSE
rmse_mlr_rec = np.sqrt(mean_squared_error(y_test_rec, y_pred_mlr_rec))
r2_mlr_rec = r2_score(y_test_rec, y_pred_mlr_rec)

print(f"Reception MLR - R²: {r2_mlr_rec:.4f}, RMSE: {rmse_mlr_rec:.4f}")
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              Reception   R-squared:                       0.598
Model:                            OLS   Adj. R-squared:                  0.584
Method:                 Least Squares   F-statistic:                     43.61
Date:                Mon, 13 Apr 2026   Prob (F-statistic):           3.20e-
82
Time:                        20:28:08   Log-Likelihood:                -
1033.2
No. Observations:                 487   AIC:                             2100.
Df Residuals:                     470   BIC:                             2172.
Df Model:                          16
Covariance Type:            nonrobust
==========================================================================================
                                          coef    std err          t      P>|t|      [0.025
------------------------------------------------------------------------------------------
const                                  23.5789      6.398      3.685      0.000      11.007
Born in uk                             -0.0309      0.019     -1.590      0.112      -
0.069       0.007
Indices of Deprivation                  0.0598      0.022      2.711      0.007       0.016
White population                       -0.0047      0.013     -0.353      0.724      -
0.031       0.021
Median age                             -0.0469      0.056     -0.835      0.404      -
0.157       0.063
Air emissions                           0.0133      0.019      0.691      0.490      -
0.024       0.051
Public transport accessibility          0.3570      0.168      2.126      0.034       0.027
```

```
Children in single-parent household     0.1617      0.045      3.615      0.000       0.074
Higher education qualifications        -0.1036      0.016     -6.598      0.000      -
0.134      -0.073
People in bad health                   -0.1229      0.160     -0.768      0.443      -
0.437       0.191
Median house price                   1.137e-06   1.46e-06      0.780      0.436   -
1.73e-06       4e-06
Population density                  -2.101e-05   3.64e-05     -0.577      0.564   -
9.26e-05    5.05e-05
Aged 0-15                              -0.0515      0.047     -1.088      0.277      -
0.144       0.042
Life expectancy                        -0.1246      0.064     -1.932      0.054      -
0.251       0.002
Crime rate                              0.0031      0.003      1.055      0.292      -
0.003       0.009
Green deficit                           0.0032      0.003      0.945      0.345      -
0.003       0.010
Well-being score                        0.0413      0.321      0.129      0.898      -
0.589       0.672
==============================================================================
Omnibus:                       14.998   Durbin-Watson:                   2.012
Prob(Omnibus):                  0.001   Jarque-Bera (JB):               16.054
Skew:                           0.382   Prob(JB):                     0.000326
Kurtosis:                       3.457   Cond. No.                     2.30e+07
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.3e+07. This might indicate that there are
strong multicollinearity or other numerical problems.
Reception MLR - R²: 0.5735, RMSE: 2.1169
```

The Reception MLR model achieves a test R^2 of 0.574 (RMSE=2.117), explaining 57.4% of variance in ward-level obesity prevalence. Statistically significant predictors ($p<0.05$) include Indices of Deprivation (=0.060), Children in single-parent household (=0.162), and Public transport accessibility (=0.357) as positive drivers, while Higher education qualifications (=-0.116) and Aged 0-15 (=-0.208) show protective negative associations.

For Year 6

```python
y_yr6 = df['Year 6']

mlr_data_yr6 = pd.concat([X, y_yr6], axis=1).dropna()
X_clean_yr6 = mlr_data_yr6[final_features]
y_clean_yr6 = mlr_data_yr6['Year 6']

X_train_yr6, X_test_yr6, y_train_yr6, y_test_yr6 = train_test_split(
    X_clean_yr6, y_clean_yr6, test_size=0.2, random_state=42
)

X_train_yr6_sm = sm.add_constant(X_train_yr6)
X_test_yr6_sm = sm.add_constant(X_test_yr6)

mlr_yr6 = sm.OLS(y_train_yr6, X_train_yr6_sm).fit()

print(mlr_yr6.summary())

y_pred_mlr_yr6 = mlr_yr6.predict(X_test_yr6_sm)

rmse_mlr_yr6 = np.sqrt(mean_squared_error(y_test_yr6, y_pred_mlr_yr6))
r2_mlr_yr6 = r2_score(y_test_yr6, y_pred_mlr_yr6)
print(f"Year 6 MLR - R²: {r2_mlr_yr6:.4f}, RMSE: {rmse_mlr_yr6:.4f}")
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                 Year 6   R-squared:                       0.637
Model:                            OLS   Adj. R-squared:                  0.624
Method:                 Least Squares   F-statistic:                     51.51
Date:                Mon, 13 Apr 2026   Prob (F-statistic):           1.62e-
92
Time:                        20:28:08   Log-Likelihood:                -
1238.2
No. Observations:                 487   AIC:                             2510.
Df Residuals:                     470   BIC:                             2582.
Df Model:                          16
Covariance Type:            nonrobust
==================================================================================================
                                        coef    std err          t      P>|t|      [0.025
--------------------------------------------------------------------------------------------------
const                                31.8196      9.745      3.265      0.001      12.670
Born in uk                           -0.0163      0.030     -0.550      0.583      -
0.074       0.042
```

```
Indices of Deprivation                  0.0849      0.034      2.524      0.012       0.019
White population                       -0.0325      0.020     -1.601      0.110      -
0.072       0.007
Median age                             -0.2256      0.086     -2.638      0.009      -
0.394      -0.058
Air emissions                           0.0321      0.029      1.098      0.273      -
0.025       0.090
Public transport accessibility          0.2936      0.256      1.148      0.252      -
0.209       0.796
Children in single-parent household     0.4013      0.068      5.891      0.000       0.267
Higher education qualifications        -0.1418      0.024     -5.929      0.000      -
0.189      -0.095
People in bad health                   -0.1871      0.244     -0.768      0.443      -
0.666       0.292
Median house price                   1.011e-06   2.22e-06      0.455      0.649   -
3.35e-06    5.38e-06
Population density                   5.026e-05   5.55e-05      0.906      0.365   -
5.87e-05       0.000
Aged 0-15                              -0.3550      0.072     -4.925      0.000      -
0.497      -0.213
Life expectancy                        -0.0578      0.098     -0.589      0.556      -
0.251       0.135
Crime rate                             -0.0035      0.005     -0.770      0.442      -
0.012       0.005
Green deficit                           0.0037      0.005      0.714      0.475      -
0.006       0.014
Well-being score                        1.0506      0.489      2.149      0.032       0.090
==============================================================================
Omnibus:                       15.782   Durbin-Watson:                   1.922
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               22.576
Skew:                           0.277   Prob(JB):                     1.25e-
05
Kurtosis:                       3.898   Cond. No.                     2.30e+07
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.3e+07. This might indicate that there are
strong multicollinearity or other numerical problems.
Year 6 MLR - R²: 0.6756, RMSE: 2.7200
```

The Year 6 MLR outperforms its Reception counterpart (test R^2=0.676, RMSE=2.720), suggesting ward-level features are more predictive of obesity at age 10-11. Significant posi-

tive predictors include Indices of Deprivation (=0.085), Children in single-parent household (=0.401), and Well-being score (=1.051), while Higher education qualifications (=-0.142), Aged 0-15 (=-0.355), and Median age (=-0.226) exhibit significant negative associations.

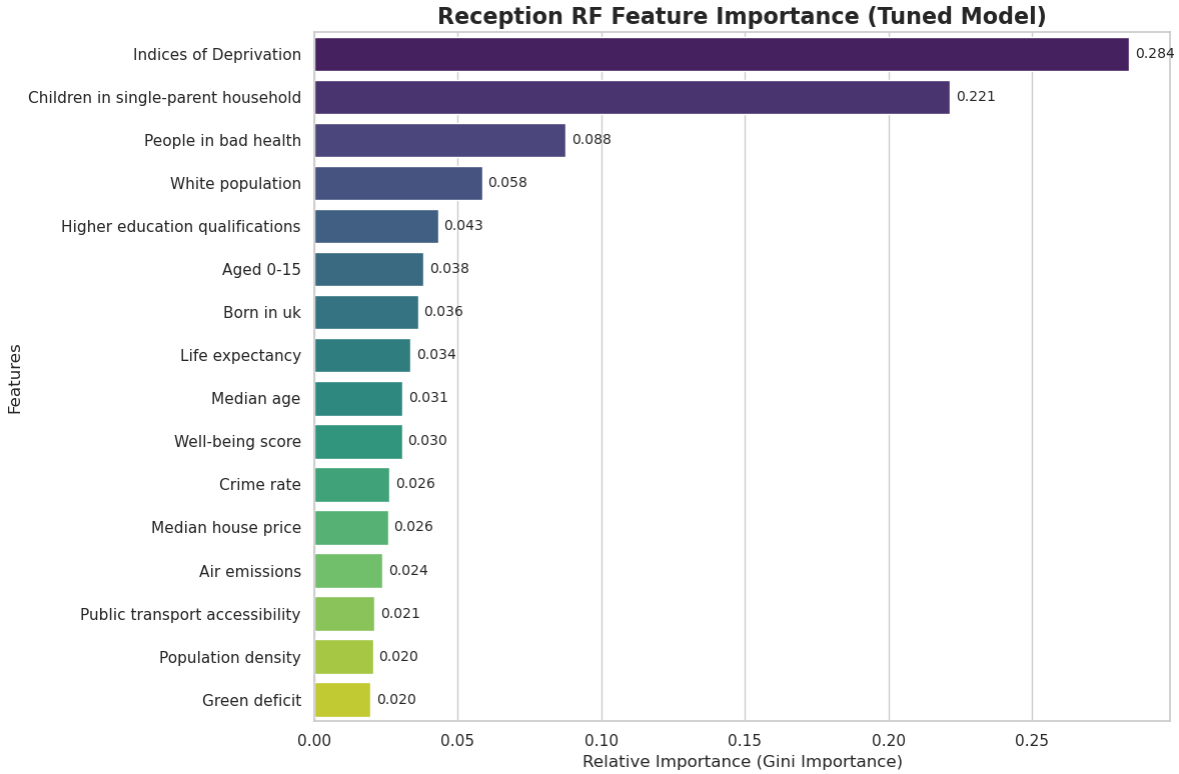
Random Forest (RF)

For Reception

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV

# Define the hyperparameter grid
param_grid = {
    'n_estimators': [100, 300, 500],
    'max_depth': [10, 15, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Initialize the base Random Forest model
rf_base = RandomForestRegressor(random_state=42, n_jobs=-1)

# Configure 5-Fold Cross-Validation
print("Reception RF - Starting CV and Hyperparameter Search...")
grid_search_rf_rec = GridSearchCV(
    estimator=rf_base,
    param_grid=param_grid,
    cv=5,
    n_jobs=-1,
    scoring='r2',
    verbose=1
)

# Fit GridSearchCV on the training data
grid_search_rf_rec.fit(X_train_rec, y_train_rec)

# Extract the best model and parameters found by CV
best_rf_rec = grid_search_rf_rec.best_estimator_

print("\nBest Parameters:")
for key, value in grid_search_rf_rec.best_params_.items():
```

```
    print(f" - {key}: {value}")

# Perform final predictions on the test set using the best model
y_pred_rf_rec = best_rf_rec.predict(X_test_rec)

# Calculate and print evaluation metrics
rmse_rf_rec = np.sqrt(mean_squared_error(y_test_rec, y_pred_rf_rec))
r2_rf_rec = r2_score(y_test_rec, y_pred_rf_rec)

print(f"Reception RF - R²: {r2_rf_rec:.4f}, RMSE: {rmse_rf_rec:.4f}")

# Plot feature importance
importances_rec = best_rf_rec.feature_importances_

importance_df_rec = pd.DataFrame({'Feature': X_train_rec.columns, 'Importance': importances_r
importance_df_rec = importance_df_rec.sort_values(by='Importance', ascending=False)

plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=importance_df_rec, hue='Feature', palette='viri

plt.title('Reception RF Feature Importance (Tuned Model)', fontsize=16, fontweight='bold')
plt.xlabel('Relative Importance (Gini Importance)', fontsize=12)
plt.ylabel('Features', fontsize=12)

# Add value labels next to the bars
for index, value in enumerate(importance_df_rec['Importance']):
    plt.text(value + 0.002, index, f'{value:.3f}', va='center', fontsize=10)

plt.tight_layout()
plt.show()
```

```
Reception RF - Starting CV and Hyperparameter Search...
Fitting 5 folds for each of 108 candidates, totalling 540 fits

Best Parameters:
 - max_depth: None
 - min_samples_leaf: 1
 - min_samples_split: 5
 - n_estimators: 300
Reception RF - R²: 0.5788, RMSE: 2.1037
```



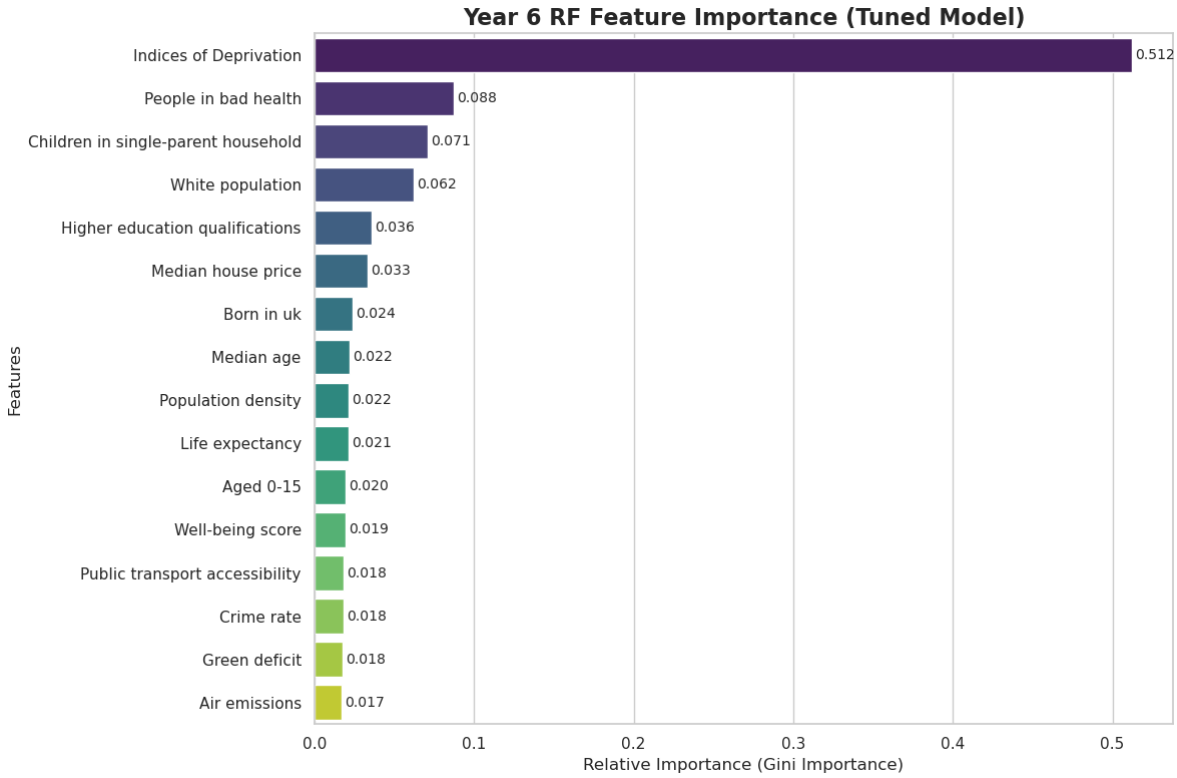
The tuned Random Forest (n_estimators=300, max_depth=None) achieves a test R^2 of 0.579 and RMSE of 2.104, a marginal improvement over MLR (ΔR^2=+0.005), suggesting nonlinear interactions contribute limited additional explanatory power for Reception.

For Year 6

```
print("Year 6 RF - Starting CV and Hyperparameter Search...")
grid_search_rf_yr6 = GridSearchCV(
    estimator=RandomForestRegressor(random_state=42, n_jobs=-1),
    param_grid=param_grid,
    cv=5,
    n_jobs=-1,
    scoring='r2',
    verbose=1
)

grid_search_rf_yr6.fit(X_train_yr6, y_train_yr6)

best_rf_yr6 = grid_search_rf_yr6.best_estimator_
```

```
print("\nBest Parameters:")
for key, value in grid_search_rf_yr6.best_params_.items():
    print(f" - {key}: {value}")

y_pred_rf_yr6 = best_rf_yr6.predict(X_test_yr6)

rmse_rf_yr6 = np.sqrt(mean_squared_error(y_test_yr6, y_pred_rf_yr6))
r2_rf_yr6 = r2_score(y_test_yr6, y_pred_rf_yr6)

print(f"Year 6 RF - R²: {r2_rf_yr6:.4f}, RMSE: {rmse_rf_yr6:.4f}")

importances_yr6 = best_rf_yr6.feature_importances_

importance_df_yr6 = pd.DataFrame({'Feature': X_train_yr6.columns, 'Importance': importances_y
importance_df_yr6 = importance_df_yr6.sort_values(by='Importance', ascending=False)

plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=importance_df_yr6, hue='Feature', palette='viri

plt.title('Year 6 RF Feature Importance (Tuned Model)', fontsize=16, fontweight='bold')
plt.xlabel('Relative Importance (Gini Importance)', fontsize=12)
plt.ylabel('Features', fontsize=12)

for index, value in enumerate(importance_df_yr6['Importance']):
    plt.text(value + 0.002, index, f'{value:.3f}', va='center', fontsize=10)

plt.tight_layout()
plt.show()
```

```
Year 6 RF - Starting CV and Hyperparameter Search...
Fitting 5 folds for each of 108 candidates, totalling 540 fits

Best Parameters:
 - max_depth: 10
 - min_samples_leaf: 2
 - min_samples_split: 2
 - n_estimators: 300
Year 6 RF - R²: 0.6822, RMSE: 2.6925
```



For Year 6, the tuned Random Forest (n_estimators=300, max_depth=10) achieves a test R^2 of 0.682 and RMSE of 2.693, again marginally improving over MLR (ΔR^2=+0.006).

XGBoost

For Reception

```
!pip install xgboost
```

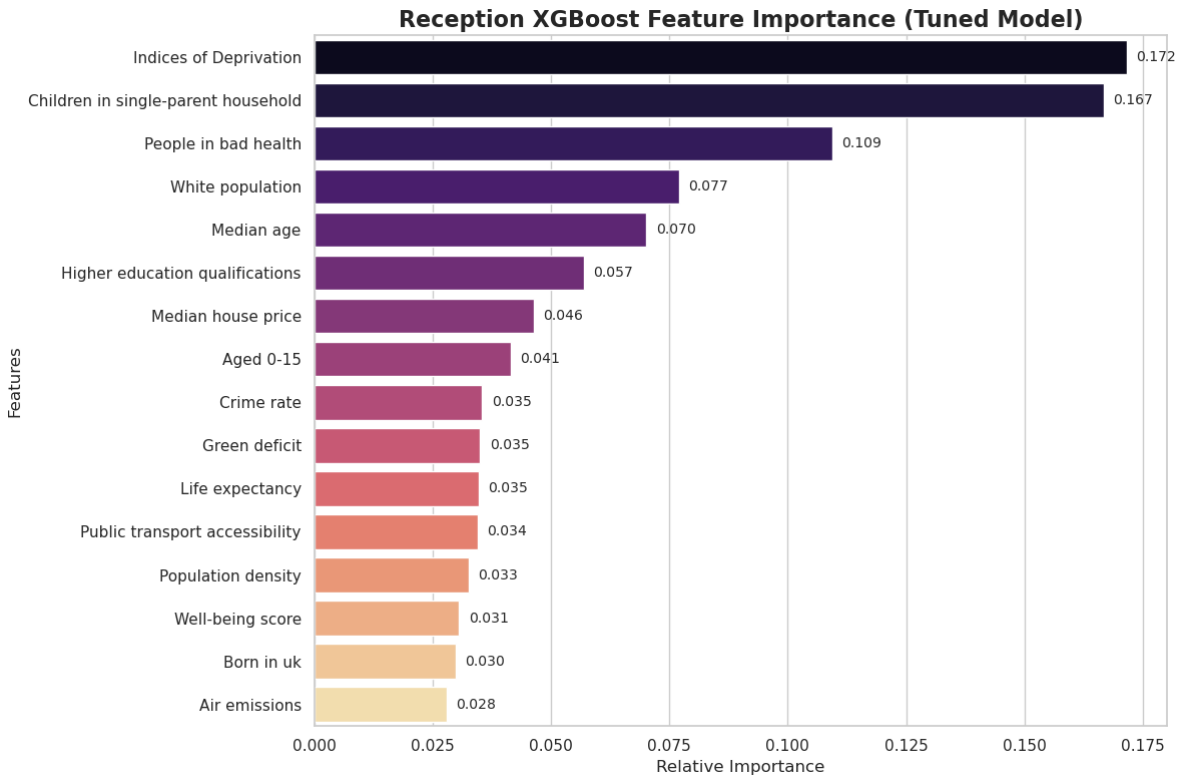
```
Collecting xgboost
  Downloading xgboost-3.2.0-py3-none-manylinux_2_28_aarch64.whl.metadata (2.1 kB)
Requirement already satisfied: numpy in /opt/conda/lib/python3.12/site-
packages (from xgboost) (2.3.3)
Collecting nvidia-nccl-cu12 (from xgboost)
  Downloading nvidia_nccl_cu12-2.29.7-py3-none-manylinux_2_18_aarch64.whl.metadata (2.1 kB)
Requirement already satisfied: scipy in /opt/conda/lib/python3.12/site-
packages (from xgboost) (1.16.2)
Downloading xgboost-3.2.0-py3-none-manylinux_2_28_aarch64.whl (131.1 MB)
```

```
                         131.1/131.1 MB 29.8 MB/s  0:00:04m0:00:0100:01
Downloading nvidia_nccl_cu12-2.29.7-py3-none-manylinux_2_18_aarch64.whl (293.6 MB)
                         293.6/293.6 MB 30.4 MB/s  0:00:09m0:00:0100:01
Installing collected packages: nvidia-nccl-cu12, xgboost
                         2/2 [xgboost]m1/2 [xgboost]
Successfully installed nvidia-nccl-cu12-2.29.7 xgboost-3.2.0
```

```python
import xgboost as xgb

# Define the hyperparameter search grid for XGBoost
param_grid_xgb = {
    'n_estimators': [100, 300, 500],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

# Initialize the base XGBoost model
xgb_base = xgb.XGBRegressor(random_state=42, n_jobs=-1, objective='reg:squarederror')

# Configure 5-Fold Cross-Validation
print("Reception XGBoost - Starting CV and Hyperparameter Search...")
grid_search_xgb_rec = GridSearchCV(
    estimator=xgb_base,
    param_grid=param_grid_xgb,
    cv=5,
    n_jobs=-1,
    scoring='r2',
    verbose=1
)

# Fit GridSearchCV on the training data
grid_search_xgb_rec.fit(X_train_rec, y_train_rec)

# Extract the best model and optimal parameters
best_xgb_rec = grid_search_xgb_rec.best_estimator_

print("\nBest Parameters:")
for key, value in grid_search_xgb_rec.best_params_.items():
    print(f" - {key}: {value}")
```

```python
# Test set prediction and evaluation
y_pred_xgb_rec = best_xgb_rec.predict(X_test_rec)

rmse_xgb_rec = np.sqrt(mean_squared_error(y_test_rec, y_pred_xgb_rec))
r2_xgb_rec = r2_score(y_test_rec, y_pred_xgb_rec)

print(f"Reception XGBoost - R²: {r2_xgb_rec:.4f}, RMSE: {rmse_xgb_rec:.4f}")

# Plot XGBoost feature importance
importances_xgb_rec = best_xgb_rec.feature_importances_

importance_df_xgb_rec = pd.DataFrame({'Feature': X_train_rec.columns, 'Importance': importanc
importance_df_xgb_rec = importance_df_xgb_rec.sort_values(by='Importance', ascending=False)

plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=importance_df_xgb_rec, hue='Feature', palette=

plt.title('Reception XGBoost Feature Importance (Tuned Model)', fontsize=16, fontweight='bold
plt.xlabel('Relative Importance', fontsize=12)
plt.ylabel('Features', fontsize=12)

for index, value in enumerate(importance_df_xgb_rec['Importance']):
    plt.text(value + 0.002, index, f'{value:.3f}', va='center', fontsize=10)

plt.tight_layout()
plt.show()
```

```
Reception XGBoost - Starting CV and Hyperparameter Search...
Fitting 5 folds for each of 108 candidates, totalling 540 fits

Best Parameters:
 - colsample_bytree: 0.8
 - learning_rate: 0.05
 - max_depth: 5
 - n_estimators: 100
 - subsample: 0.8
Reception XGBoost - R²: 0.5800, RMSE: 2.1006
```



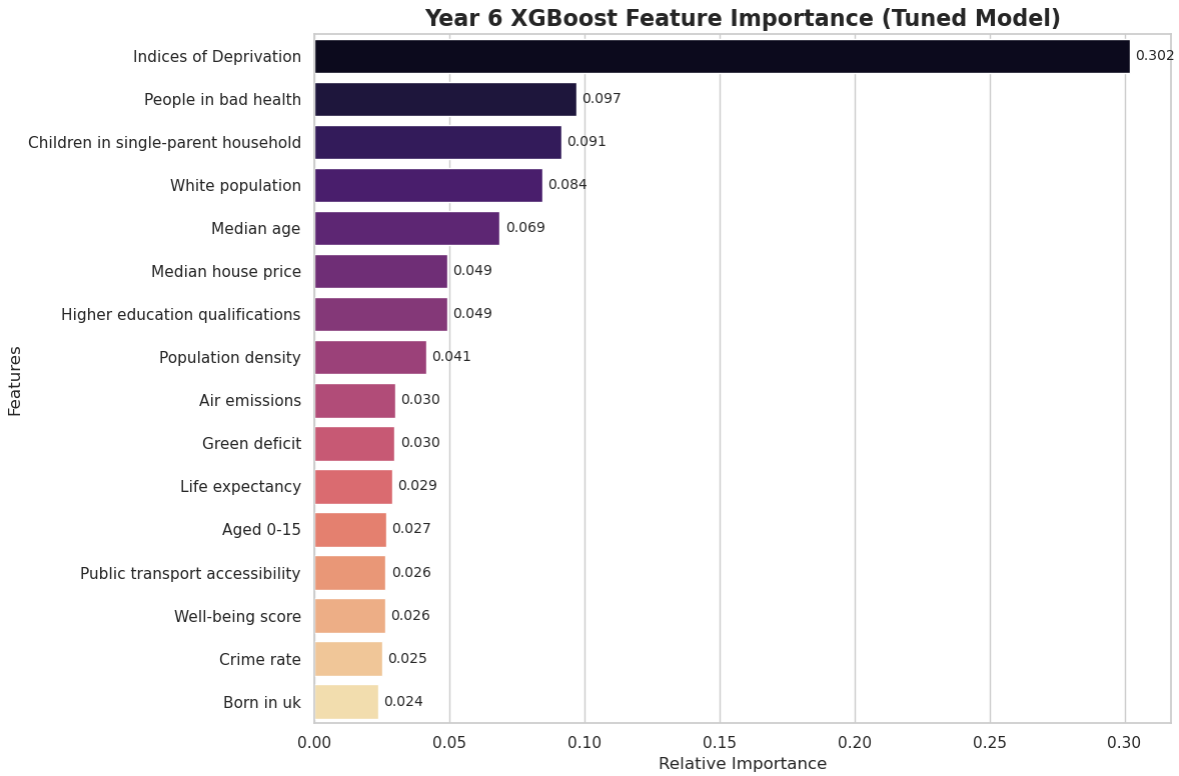
XGBoost achieves the best Reception performance (R^2=0.580, RMSE=2.101) with an optimal configuration of n_estimators=100, max_depth=5, learning_rate=0.05. As the best-performing model, it is selected as the basis for SHAP interpretation.

For Year 6

```
print("Year 6 XGBoost - Starting CV and Hyperparameter Search...")
grid_search_xgb_yr6 = GridSearchCV(
    estimator=xgb.XGBRegressor(random_state=42, n_jobs=-1, objective='reg:squarederror'),
    param_grid=param_grid_xgb,
    cv=5,
    n_jobs=-1,
    scoring='r2',
    verbose=1
)

grid_search_xgb_yr6.fit(X_train_yr6, y_train_yr6)

best_xgb_yr6 = grid_search_xgb_yr6.best_estimator_
```

```
print("\nBest Parameters:")
for key, value in grid_search_xgb_yr6.best_params_.items():
    print(f" - {key}: {value}")

y_pred_xgb_yr6 = best_xgb_yr6.predict(X_test_yr6)

rmse_xgb_yr6 = np.sqrt(mean_squared_error(y_test_yr6, y_pred_xgb_yr6))
r2_xgb_yr6 = r2_score(y_test_yr6, y_pred_xgb_yr6)

print(f"Year 6 XGBoost - R²: {r2_xgb_yr6:.4f}, RMSE: {rmse_xgb_yr6:.4f}")

importances_xgb_yr6 = best_xgb_yr6.feature_importances_

importance_df_xgb_yr6 = pd.DataFrame({'Feature': X_train_yr6.columns, 'Importance': importanc
importance_df_xgb_yr6 = importance_df_xgb_yr6.sort_values(by='Importance', ascending=False)

plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=importance_df_xgb_yr6, hue='Feature', palette=

plt.title('Year 6 XGBoost Feature Importance (Tuned Model)', fontsize=16, fontweight='bold')
plt.xlabel('Relative Importance', fontsize=12)
plt.ylabel('Features', fontsize=12)

for index, value in enumerate(importance_df_xgb_yr6['Importance']):
    plt.text(value + 0.002, index, f'{value:.3f}', va='center', fontsize=10)

plt.tight_layout()
plt.show()
```

```
Year 6 XGBoost - Starting CV and Hyperparameter Search...
Fitting 5 folds for each of 108 candidates, totalling 540 fits

Best Parameters:
 - colsample_bytree: 0.8
 - learning_rate: 0.01
 - max_depth: 5
 - n_estimators: 500
 - subsample: 0.8
Year 6 XGBoost - R²: 0.6831, RMSE: 2.6887
```



For Year 6, XGBoost again achieves the highest performance (R^2=0.683, RMSE=2.689), with n_estimators=500, max_depth=5, learning_rate=0.01. It is similarly selected for subsequent SHAP analysis.

Model Performance Comparison

```
from IPython.display import display

#   Reception: Model Performance Comparison
results_rec = pd.DataFrame({
    'Model': ['Multiple Linear Regression (MLR)', 'Random Forest (Tuned)', 'XGBoost (Tuned)']
    'R-squared (R²)': [r2_mlr_rec, r2_rf_rec, r2_xgb_rec],
    'RMSE': [rmse_mlr_rec, rmse_rf_rec, rmse_xgb_rec],
    'Model Type': ['Linear / Baseline', 'Non-linear / Ensemble (Bagging)', 'Non-linear / Ens
})

# Round for readability
results_rec = results_rec.round({'R-squared (R²)': 4, 'RMSE': 4})

print("Reception: Model Performance Comparison")
```

```
display(results_rec)

#   Year 6: Model Performance Comparison
results_yr6 = pd.DataFrame({
    'Model': ['Multiple Linear Regression (MLR)', 'Random Forest (Tuned)', 'XGBoost (Tuned)']
    'R-squared (R²)': [r2_mlr_yr6, r2_rf_yr6, r2_xgb_yr6],
    'RMSE': [rmse_mlr_yr6, rmse_rf_yr6, rmse_xgb_yr6],
    'Model Type': ['Linear / Baseline', 'Non-linear / Ensemble (Bagging)', 'Non-linear / Ense
})

results_yr6 = results_yr6.round({'R-squared (R²)': 4, 'RMSE': 4})

print("Year 6: Model Performance Comparison")
display(results_yr6)

#   Combined comparison across both cohorts (for Results & Discussion)
# This table merges both cohorts side by side for easy cross-group comparison
results_combined = pd.DataFrame({
    'Model': ['Multiple Linear Regression (MLR)', 'Random Forest (Tuned)', 'XGBoost (Tuned)']
    'Reception R²': [r2_mlr_rec, r2_rf_rec, r2_xgb_rec],
    'Reception RMSE': [rmse_mlr_rec, rmse_rf_rec, rmse_xgb_rec],
    'Year 6 R²': [r2_mlr_yr6, r2_rf_yr6, r2_xgb_yr6],
    'Year 6 RMSE': [rmse_mlr_yr6, rmse_rf_yr6, rmse_xgb_yr6],
})

results_combined = results_combined.round(4)

print("Combined: Reception vs Year 6")
display(results_combined)
```

```
Reception: Model Performance Comparison
```

	Model	R-squared (R^2)	RMSE	Model Type
0	Multiple Linear Regression (MLR)	0.5735	2.1169	Linear / Baseline
1	Random Forest (Tuned)	0.5788	2.1037	Non-linear / Ensemble (Bagging)
2	XGBoost (Tuned)	0.5800	2.1006	Non-linear / Ensemble (Boosting)

```
Year 6: Model Performance Comparison
```

	Model	R-squared (R^2)	RMSE	Model Type
0	Multiple Linear Regression (MLR)	0.6756	2.7200	Linear / Baseline
1	Random Forest (Tuned)	0.6822	2.6925	Non-linear / Ensemble (Bagging)
2	XGBoost (Tuned)	0.6831	2.6887	Non-linear / Ensemble (Boosting)

```
Combined: Reception vs Year 6
```

	Model	Reception R^2	Reception RMSE	Year 6 R^2	Year 6 RMSE
0	Multiple Linear Regression (MLR)	0.5735	2.1169	0.6756	2.7200
1	Random Forest (Tuned)	0.5788	2.1037	0.6822	2.6925
2	XGBoost (Tuned)	0.5800	2.1006	0.6831	2.6887

Across both cohorts, Year 6 obesity is consistently more predictable than Reception (R^2 0.68 vs 0.58), suggesting that ward-level features accumulate greater explanatory power as children age, while Reception obesity may be more strongly driven by individual-level factors not captured in area-level data. Within each cohort, performance gains from MLR to XGBoost are consistent but modest ($\Delta R^2 < 0.01$).

Results and discussion

[go back to the top]

```
display(results_combined)
```

	Model	Reception R^2	Reception RMSE	Year 6 R^2	Year 6 RMSE
0	Multiple Linear Regression (MLR)	0.5735	2.1169	0.6756	2.7200
1	Random Forest (Tuned)	0.5788	2.1037	0.6822	2.6925
2	XGBoost (Tuned)	0.5800	2.1006	0.6831	2.6887

XGBoost achieves the highest performance for both cohorts and is therefore selected as the basis for SHAP interpretation to address RQ2.

SHAP

```
!pip install shap
```

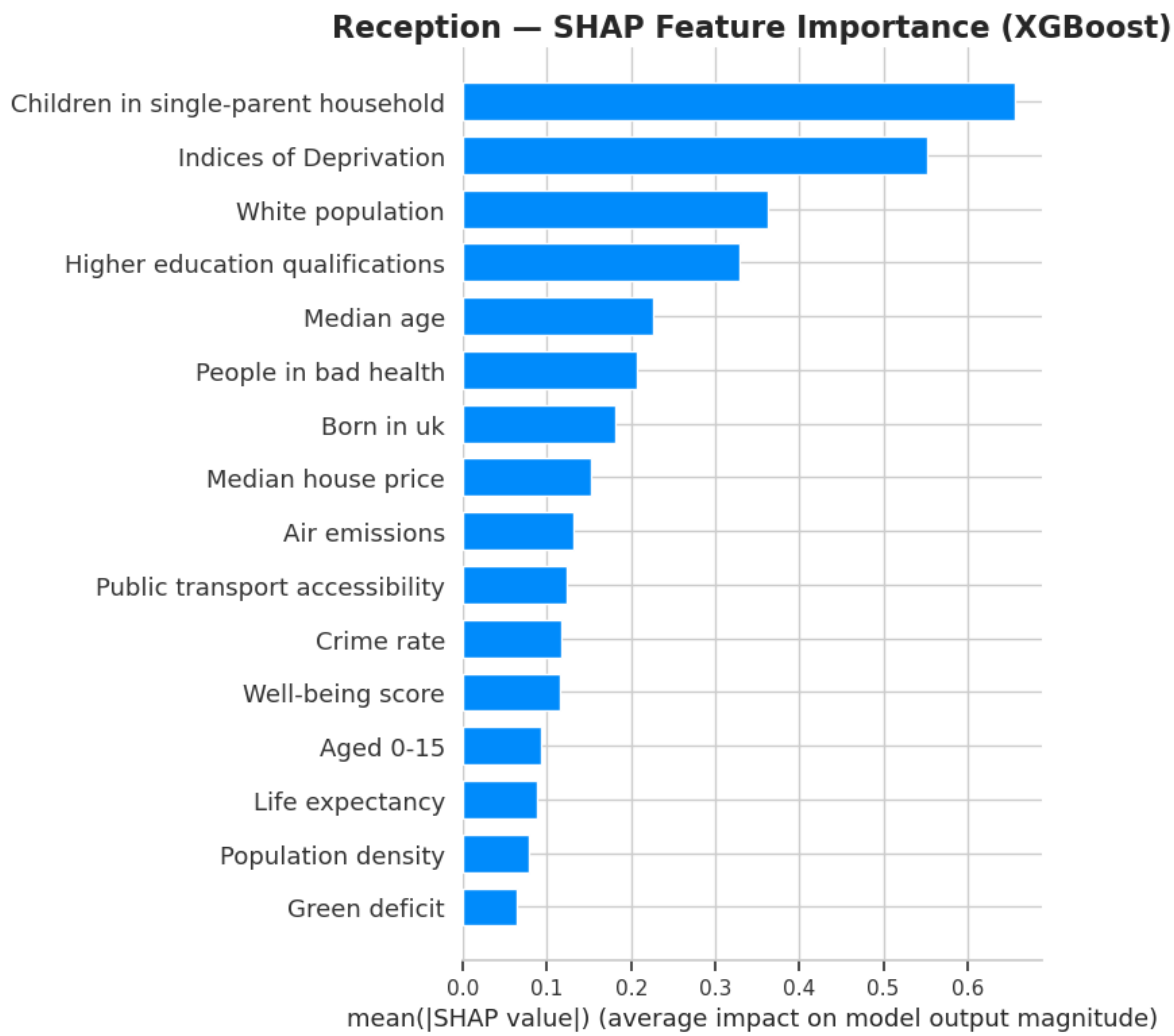
```
Collecting shap
  Downloading shap-0.51.0-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_
Requirement already satisfied: numpy>=2 in /opt/conda/lib/python3.12/site-
packages (from shap) (2.3.3)
Requirement already satisfied: scipy in /opt/conda/lib/python3.12/site-
packages (from shap) (1.16.2)
Requirement already satisfied: scikit-learn in /opt/conda/lib/python3.12/site-
packages (from shap) (1.7.2)
Requirement already satisfied: pandas in /opt/conda/lib/python3.12/site-
packages (from shap) (2.3.2)
Requirement already satisfied: tqdm>=4.27.0 in /opt/conda/lib/python3.12/site-
packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /opt/conda/lib/python3.12/site-
packages (from shap) (25.0)
Collecting slicer==0.0.8 (from shap)
  Downloading slicer-0.0.8-py3-none-any.whl.metadata (4.0 kB)
Collecting numba (from shap)
  Downloading numba-0.65.0-cp312-cp312-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl.meta
Collecting llvmlite (from shap)
  Downloading llvmlite-0.47.0-cp312-cp312-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl.m
Collecting cloudpickle (from shap)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: typing-extensions in /opt/conda/lib/python3.12/site-
packages (from shap) (4.14.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/python3.12/site-
packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.12/site-
packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /opt/conda/lib/python3.12/site-
packages (from pandas->shap) (2025.2)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-
packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in /opt/conda/lib/python3.12/site-
packages (from scikit-learn->shap) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/conda/lib/python3.12/site-
packages (from scikit-learn->shap) (3.6.0)
Downloading shap-0.51.0-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_
                         1.1/1.1 MB 33.8 MB/s  0:00:00
Downloading slicer-0.0.8-py3-none-any.whl (15 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
```

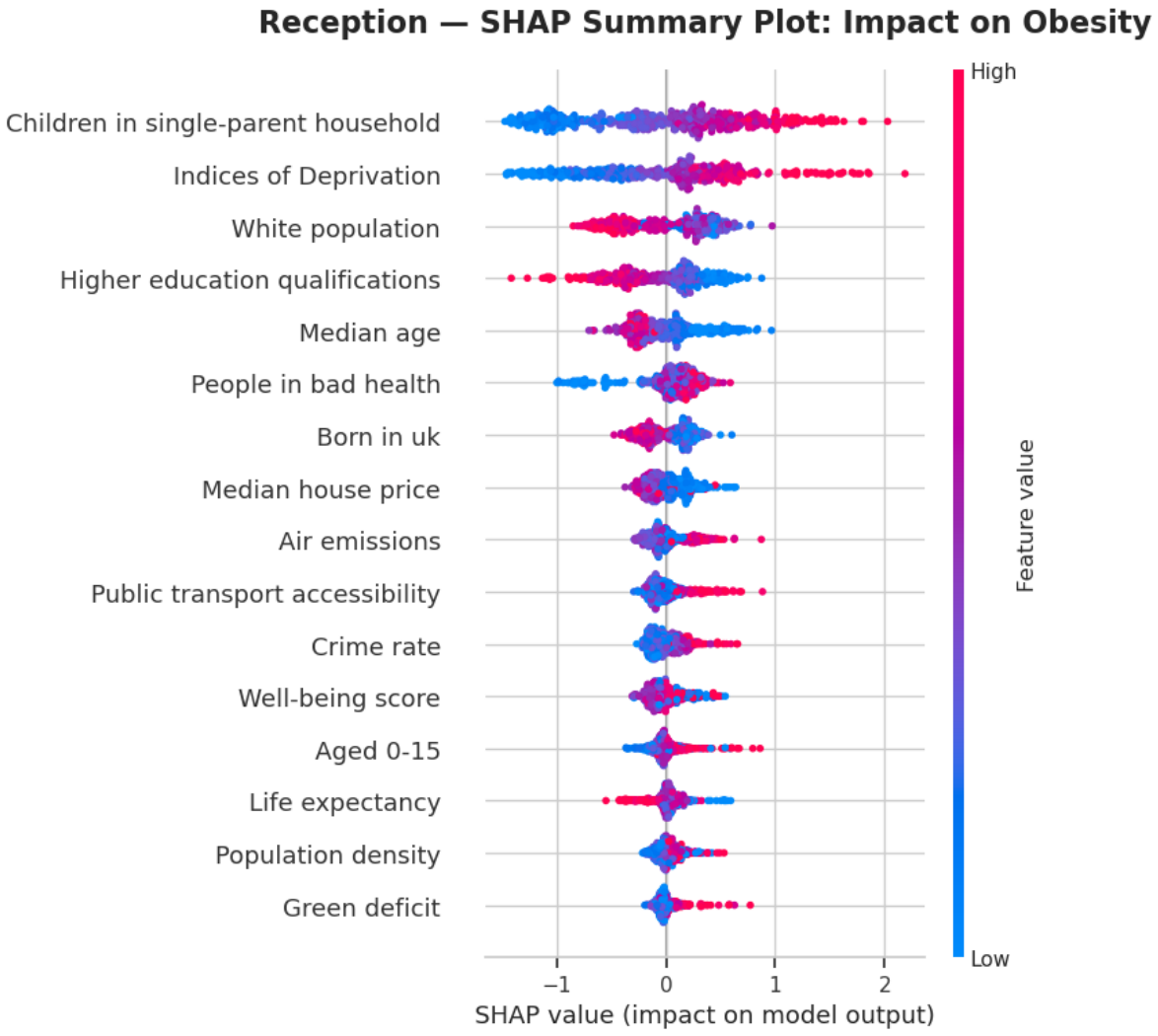
```
Downloading llvmlite-0.47.0-cp312-cp312-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl (55
                         55.1/55.1 MB 32.9 MB/s  0:00:01 eta 0:00:01
Downloading numba-0.65.0-cp312-cp312-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl (3.5 M
                         3.5/3.5 MB 18.2 MB/s  0:00:00 eta 0:00:01
Installing collected packages: slicer, llvmlite, cloudpickle, numba, shap
                         5/5 [shap][32m3/5 [numba]te]
Successfully installed cloudpickle-3.1.2 llvmlite-0.47.0 numba-0.65.0 shap-
0.51.0 slicer-0.0.8
```

For Reception

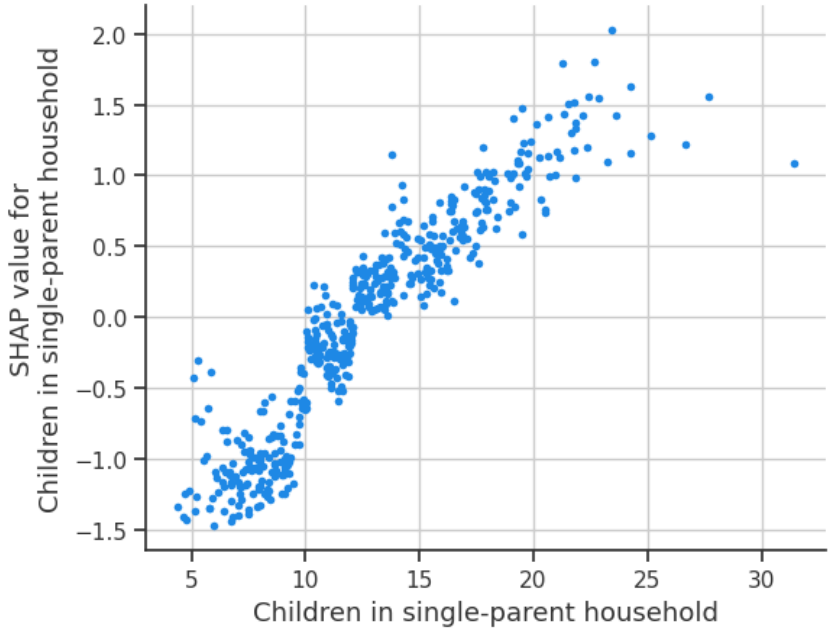
```python
import shap
# I asked Claude about how to write SHAP codes
# Initialise TreeExplainer with the best XGBoost model for Reception
explainer_rec = shap.TreeExplainer(best_xgb_rec)

# Compute SHAP values on the training set for global interpretation
shap_values_rec = explainer_rec.shap_values(X_train_rec)

# Figure 1: Global feature importance bar plot
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values_rec, X_train_rec, plot_type="bar", show=False)
plt.title("Reception - SHAP Feature Importance (XGBoost)", fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

# Figure 2: Beeswarm plot - shows direction and magnitude of each feature's impact
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values_rec, X_train_rec, show=False)
plt.title("Reception - SHAP Summary Plot: Impact on Obesity", fontsize=16, fontweight='bold'
plt.tight_layout()
plt.show()

# Figure 3: Dependence plots for the top 2 features - explore nonlinear marginal effects
# Update feature names below if Reception's top features differ from Year 6
top_feature_1_rec = 'Children in single-parent household'
top_feature_2_rec = 'Indices of Deprivation'

shap.dependence_plot(top_feature_1_rec, shap_values_rec, X_train_rec, interaction_index=None
plt.title(f"Reception - SHAP Dependence Plot: {top_feature_1_rec}", fontsize=14, fontweight=
plt.tight_layout()
plt.show()
```

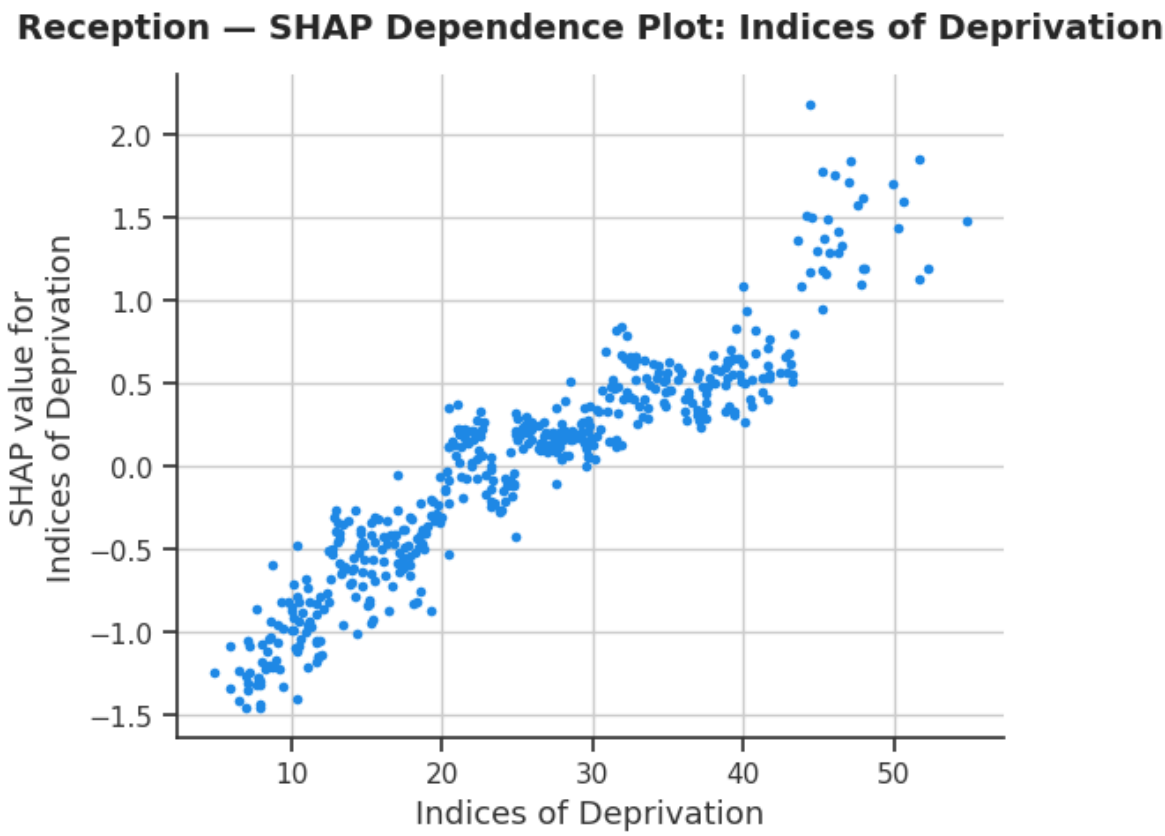
```
shap.dependence_plot(top_feature_2_rec, shap_values_rec, X_train_rec, interaction_index=None
plt.title(f"Reception - SHAP Dependence Plot: {top_feature_2_rec}", fontsize=14, fontweight=
plt.tight_layout()
plt.show()
```




Reception — SHAP Summary Plot: Impact on Obesity
Children in single-parent household
Indices of Deprivation
White population
Higher education qualifications
Median age
People in bad health
Born in uk
Median house price
Air emissions
Public transport accessibility
Crime rate
Well-being score
Aged 0-15
Life expectancy
Population density
Green deficit
−1
0
1
2
SHAP value (impact on model output)
High
Feature value
Low

Reception — SHAP Dependence Plot: Children in single-parent household





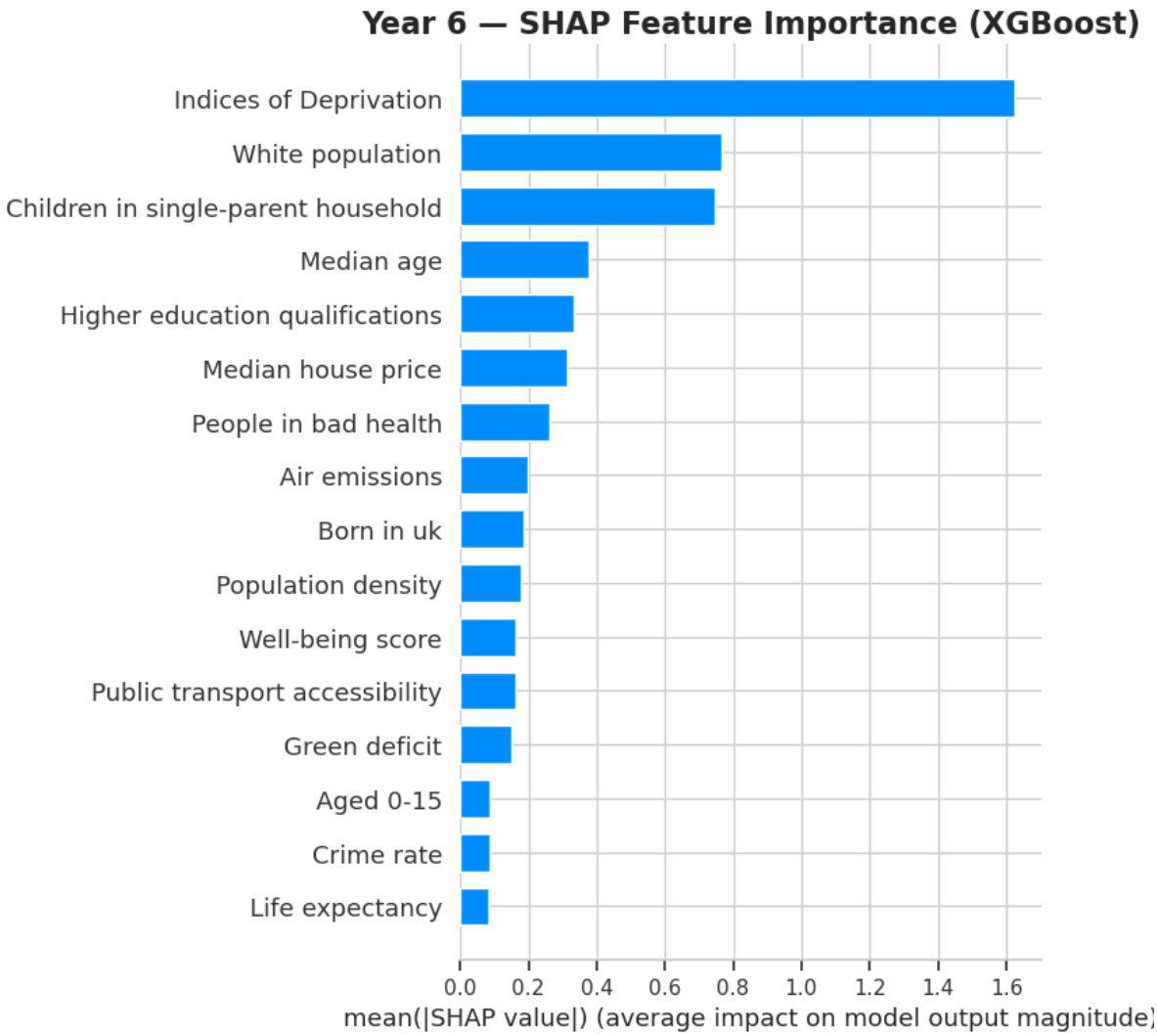
Children in single-parent household and Indices of Deprivation are the dominant drivers for Reception (mean |SHAP| 0.65 and 0.55), both positively associated with obesity. Higher education qualifications exerts a strong negative influence, reflecting the protective role of parental human capital in early childhood.

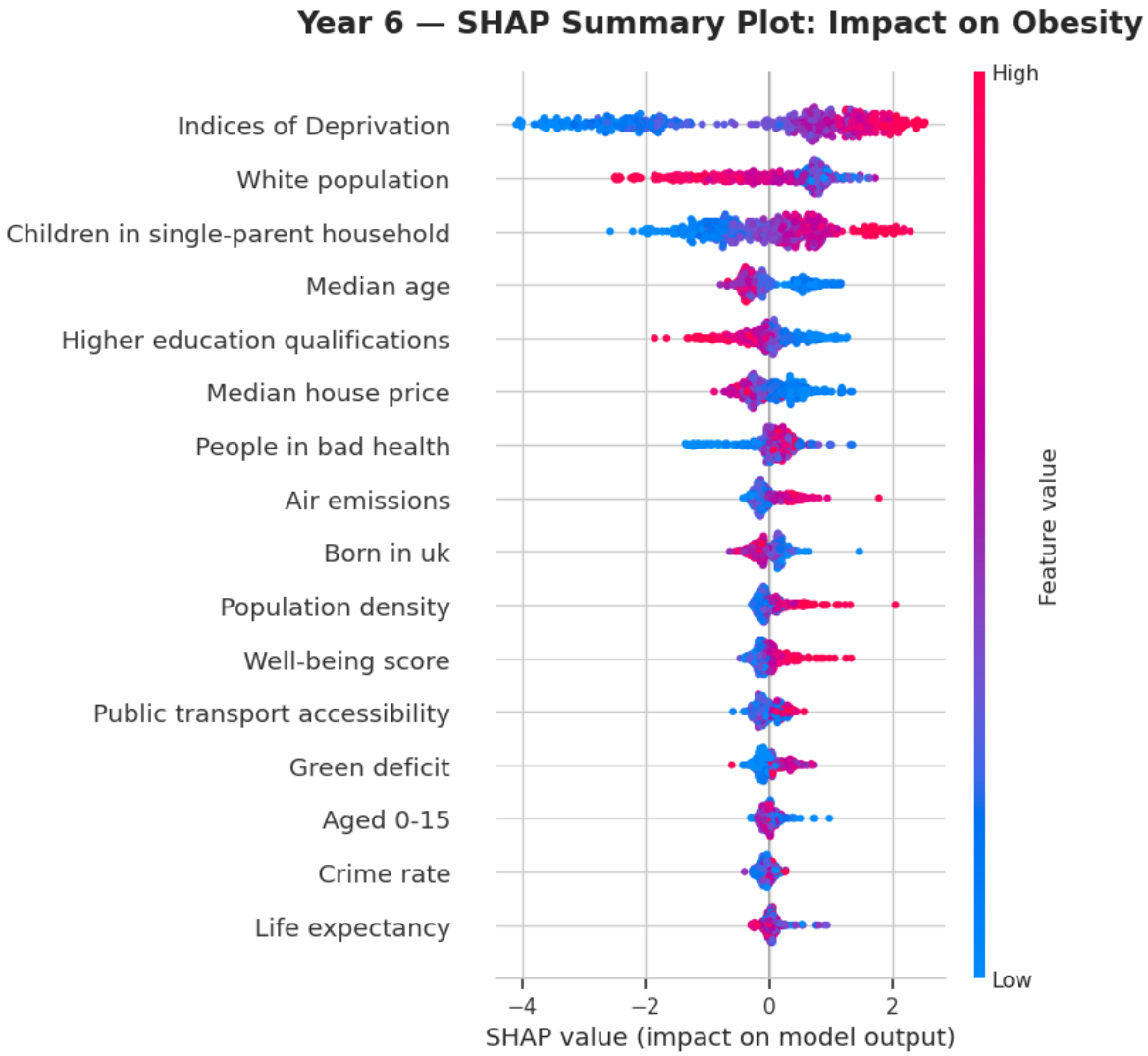
The Children in single-parent household dependence plot reveals a nonlinear positive relationship, with SHAP values turning strongly positive beyond ~15%. The Indices of Deprivation plot confirms a near-monotonic positive relationship (SHAP range: -1.5 to +2.0).

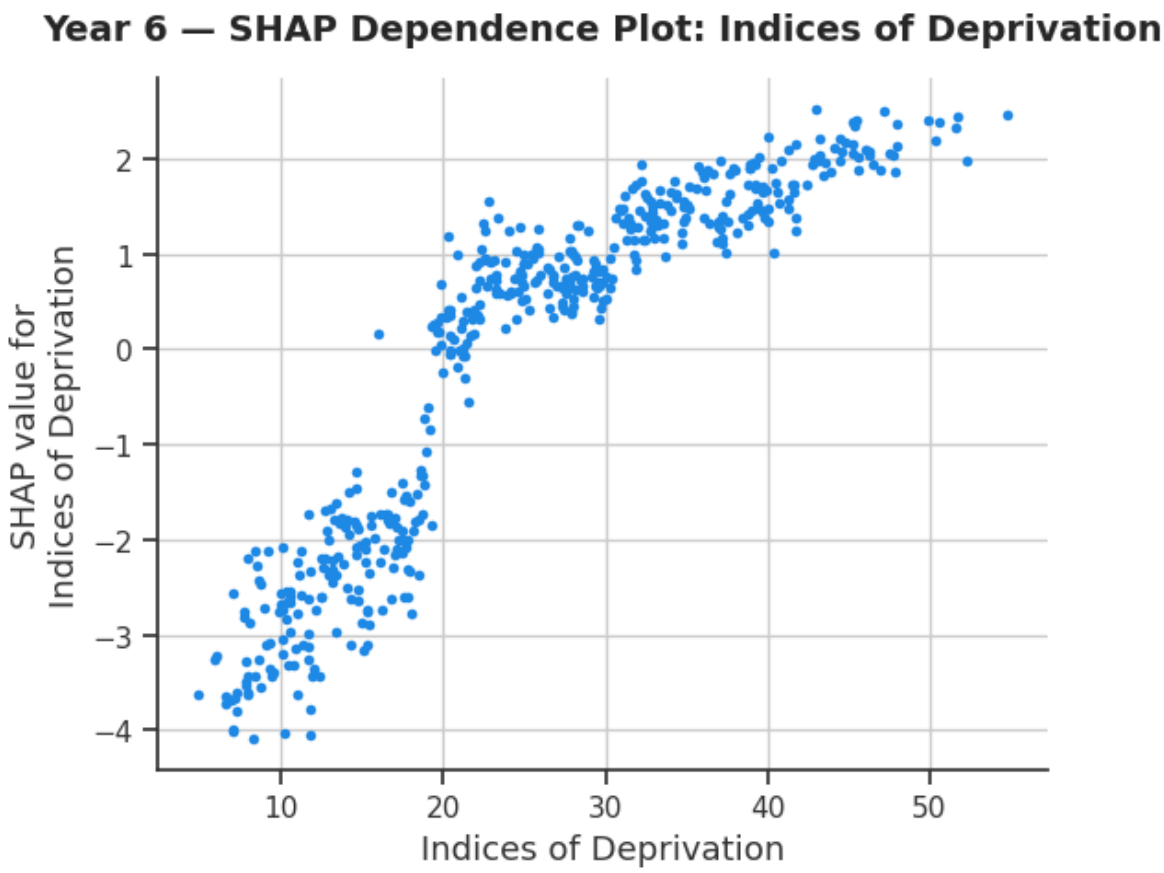
For Year 6

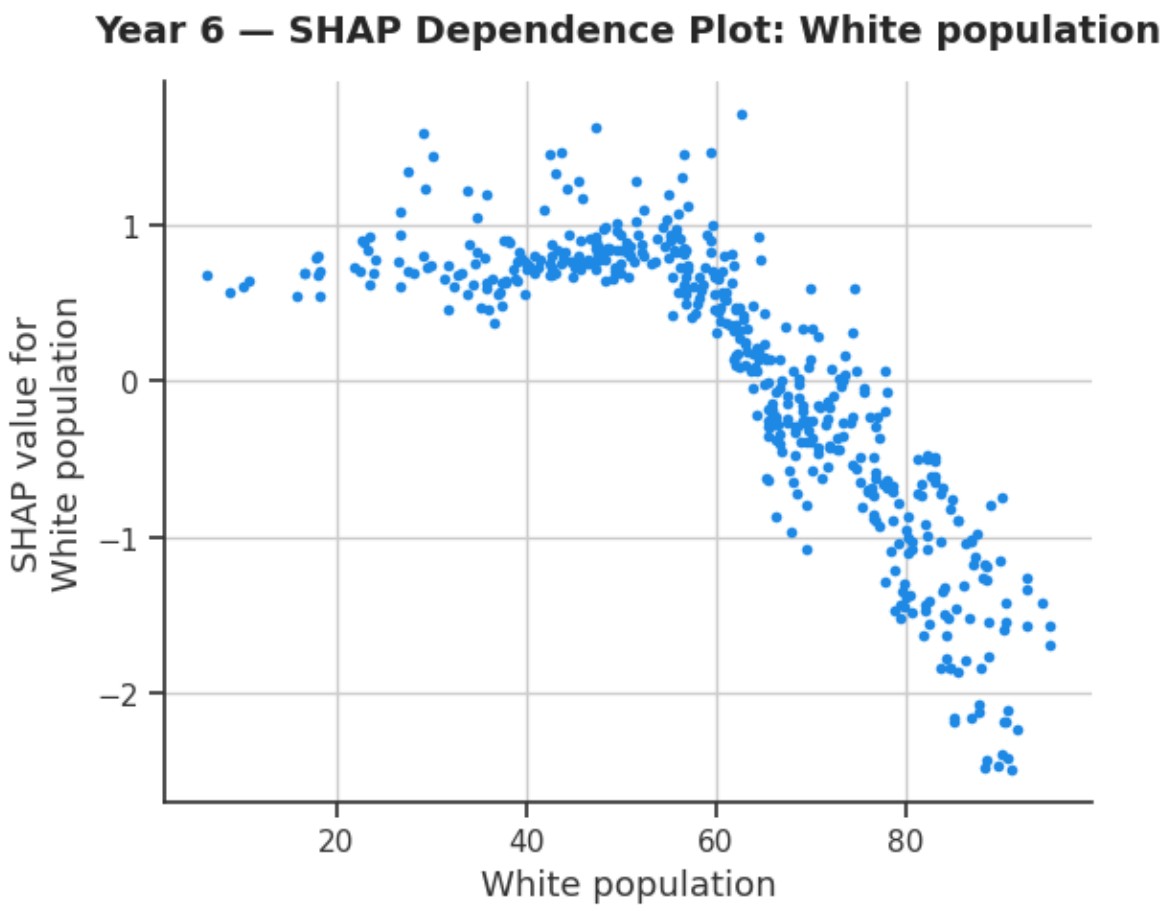
```
# Initialise TreeExplainer with the best XGBoost model for Year 6
explainer_yr6 = shap.TreeExplainer(best_xgb_yr6)

# Compute SHAP values on the training set for global interpretation
shap_values_yr6 = explainer_yr6.shap_values(X_train_yr6)

# Figure 1: Global feature importance bar plot
```

```
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values_yr6, X_train_yr6, plot_type="bar", show=False)
plt.title("Year 6 - SHAP Feature Importance (XGBoost)", fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

# Figure 2: Beeswarm plot - shows direction and magnitude of each feature's impact
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values_yr6, X_train_yr6, show=False)
plt.title("Year 6 - SHAP Summary Plot: Impact on Obesity", fontsize=16, fontweight='bold', pa
plt.tight_layout()
plt.show()

# Figure 3: Dependence plots for the top 2 features - explore nonlinear marginal effects
# Update feature names below after inspecting the bar plot above
top_feature_1_yr6 = 'Indices of Deprivation'
top_feature_2_yr6 = 'White population'

shap.dependence_plot(top_feature_1_yr6, shap_values_yr6, X_train_yr6, interaction_index=None
plt.title(f"Year 6 - SHAP Dependence Plot: {top_feature_1_yr6}", fontsize=14, fontweight='bol
plt.tight_layout()
plt.show()

shap.dependence_plot(top_feature_2_yr6, shap_values_yr6, X_train_yr6, interaction_index=None
plt.title(f"Year 6 - SHAP Dependence Plot: {top_feature_2_yr6}", fontsize=14, fontweight='bol
plt.tight_layout()
plt.show()
```


Year 6 — SHAP Feature Importance (XGBoost)
Indices of Deprivation
White population
Children in single-parent household
Median age
Higher education qualifications
Median house price
People in bad health
Air emissions
Born in uk
Population density
Well-being score
Public transport accessibility
Green deficit
Aged 0-15
Crime rate
Life expectancy
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
mean(|SHAP value|) (average impact on model output magnitude)


Year 6 — SHAP Summary Plot: Impact on Obesity
Indices of Deprivation
White population
Children in single-parent household
Median age
Higher education qualifications
Median house price
People in bad health
Air emissions
Born in uk
Population density
Well-being score
Public transport accessibility
Green deficit
Aged 0-15
Crime rate
Life expectancy
−4
−2
0
2
SHAP value (impact on model output)
High
Feature value
Low


Year 6 — SHAP Dependence Plot: Indices of Deprivation
SHAP value for
Indices of Deprivation
2
1
0
−1
−2
−3
−4
10
20
30
40
50
Indices of Deprivation



Indices of Deprivation remains the overwhelmingly dominant driver in Year 6 (mean |SHAP| 1.60, range: -4.0 to +2.5), substantially wider than Reception, consistent with cumulative neighbourhood deprivation effects over childhood. White population emerges second (0.75), showing a clear threshold: SHAP values remain positive below ~60% but drop sharply thereafter, suggesting that higher White population proportions, a proxy for socioeconomic advantage in London, exert a strong protective effect against obesity by Year 6.

Comparison and Discussion

Comparing cohorts directly addresses RQ2. Indices of Deprivation rises from second to first, with mean |SHAP| nearly tripling (0.55→1.60), consistent with a cumulative deprivation hypothesis: obesogenic neighbourhood effects compound over childhood and become increasingly detectable by Year 6.

Conversely, Children in single-parent household drops from first to third, suggesting family structure is most influential in early childhood when parental feeding practices dominate, and diminishes as children gain autonomy and become more exposed to school environments.

These findings underscore the need for stage-specific interventions: **family-focused policies in early years, and neighbourhood deprivation reduction strategies as children approach adolescence**.

Limitation

This study has several limitations. Reliance on area-level data excludes individual-level predictors such as diet and physical activity, likely explaining the moderate R^2 ceiling. The cross-sectional design also precludes causal inference. Future research should integrate individual-level records with area-level data and adopt longitudinal designs.

Conclusion

[go back to the top]

This study employed VIF, MLR, Random Forest, XGBoost, and SHAP to investigate the ward-level drivers of childhood obesity across Reception and Year 6 cohorts in London. XGBoost achieved the best predictive performance for both groups (R^2=0.580 and 0.683 respectively), with Year 6 consistently more predictable than Reception. SHAP analysis revealed a clear developmental shift in key drivers: family structure (Children in single-parent household) dominates in early childhood, while neighbourhood deprivation (Indices of Deprivation) becomes overwhelmingly influential by Year 6. These findings suggest that effective interventions should be stage-specific: prioritising family support in early years and broader deprivation reduction strategies as children age.

```
end_time = time.time()
run_time = end_time - start_time
print("Total run time: ", run_time, "seconds")
```

```
Total run time:  305.47827553749084 seconds
```

References

[go back to the top]

Colmenarejo, G. (2020) 'Machine Learning Models to Predict Childhood and Adolescent Obesity: A Review', Nutrients, 12(8). Available at: https://doi.org/10.3390/nu12082466.

National Child Measurement Programme annual report, academic year 2024 to 2025 (no date). Available at: https://fingertips.phe.org.uk/static-reports/obesity-physical-activity-nutrition/national-child-measurement-programme-2024-2025-academic-year.html (Accessed: 8 April 2026).

Pei, Z. et al. (2013) 'Early life risk factors of being overweight at 10 years of age: results of the German birth cohorts GINIplus and LISAplus', European Journal of Clinical Nutrition, 67(8), pp. 855–862. Available at: https://doi.org/10.1038/ejcn.2013.80.

Hammond R. et al. (2019) 'Predicting childhood obesity using electronic health records and publicly available data', PLOS ONE, 14(4), p. e0215571. Available at: https://doi.org/10.1371/journal.pone.0215571.

Jeong, S. et al. (2023) 'Understanding cross-data dynamics of individual and social/environmental factors through a public health lens: explainable machine learning approaches', Frontiers in Public Health, 11. Available at: https://doi.org/10.3389/fpubh.2023.1257861.

Mazza, O. et al. (2025) 'Spatial patterns of childhood obesity clusters linked to socioeconomic inequalities', Frontiers in Public Health, 13. Available at: https://doi.org/10.3389/fpubh.2025.1497090.

Salerno, P.R.V.O. et al. (2024) 'County-level socio-environmental factors and obesity prevalence in the United States', Diabetes, Obesity and Metabolism, 26(5), pp. 1766–1774. Available at: https://doi.org/10.1111/dom.15488.